



دانشگاه صنعتی امیر کبیر
(پلی تکنیک تهران)

Computer Architecture

Memory

Afarghadan@aut.ac.ir

عظیم فرقدان



اسفند ۱۴۰۳

1. Hierarchal Memory

2. Cache

3. Cache mapping

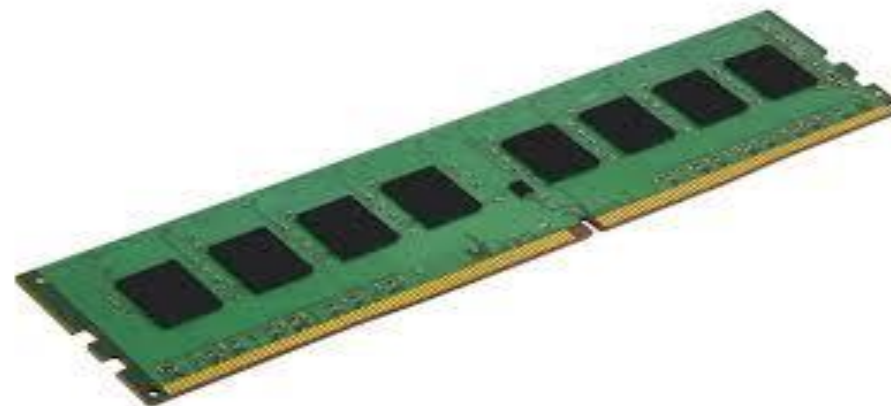
Memories



Memory Technologies

Memory technology	Typical access time	\$per GB in 2008
SRAM	0.5 – 2.5 ns	\$2000 – \$5000
DRAM	50 – 70 ns	\$20 – \$75
Magnetic disk	5000000 – 20000000 ns	\$0.2 – \$2

speed	processor	size	cost(\$/bit)	technology
Fastest	Memory	smallest	Highest	SRAM
	memory			DRAM
slowest	memory	Biggest	lowest	Magnetic disk

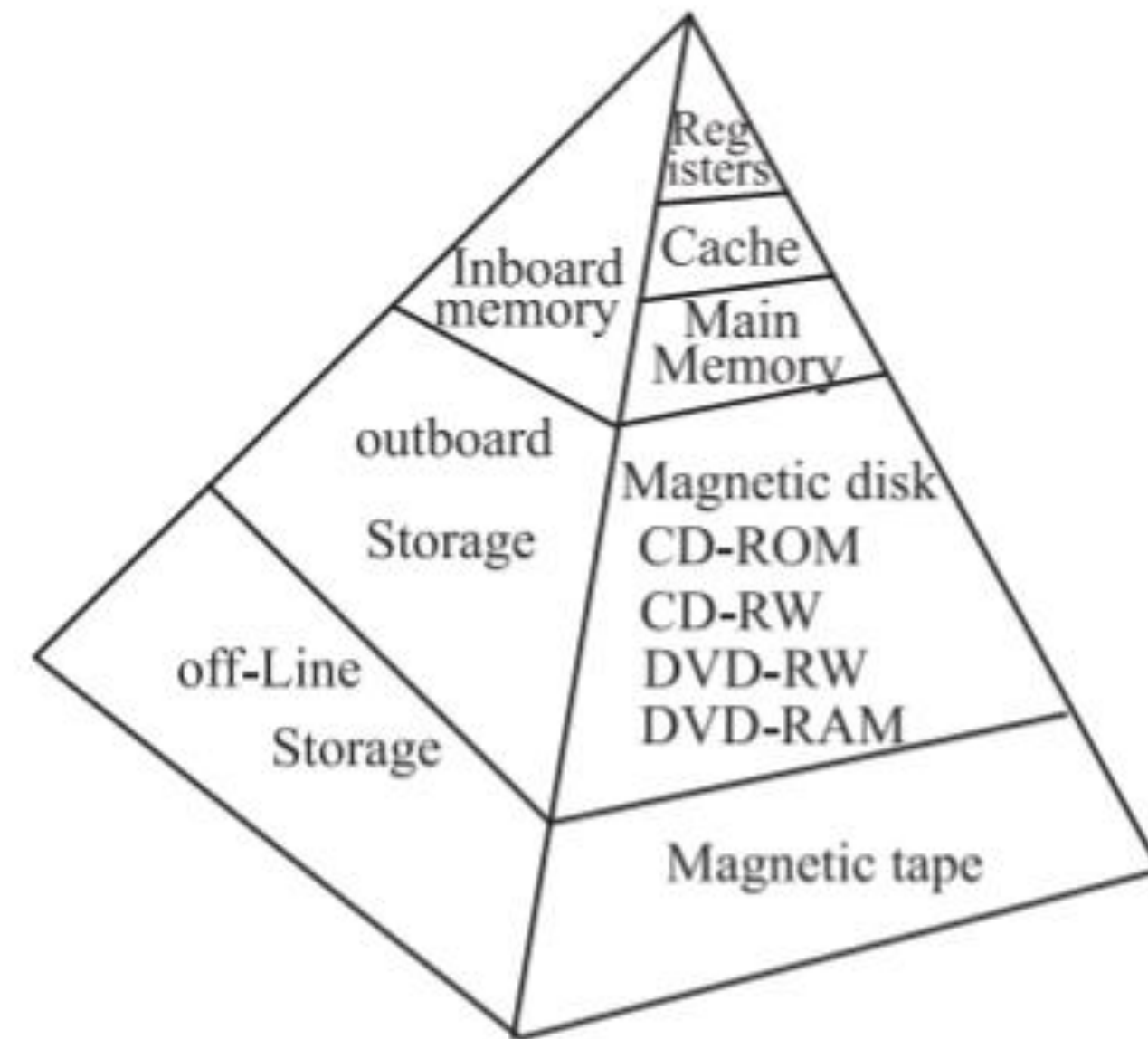
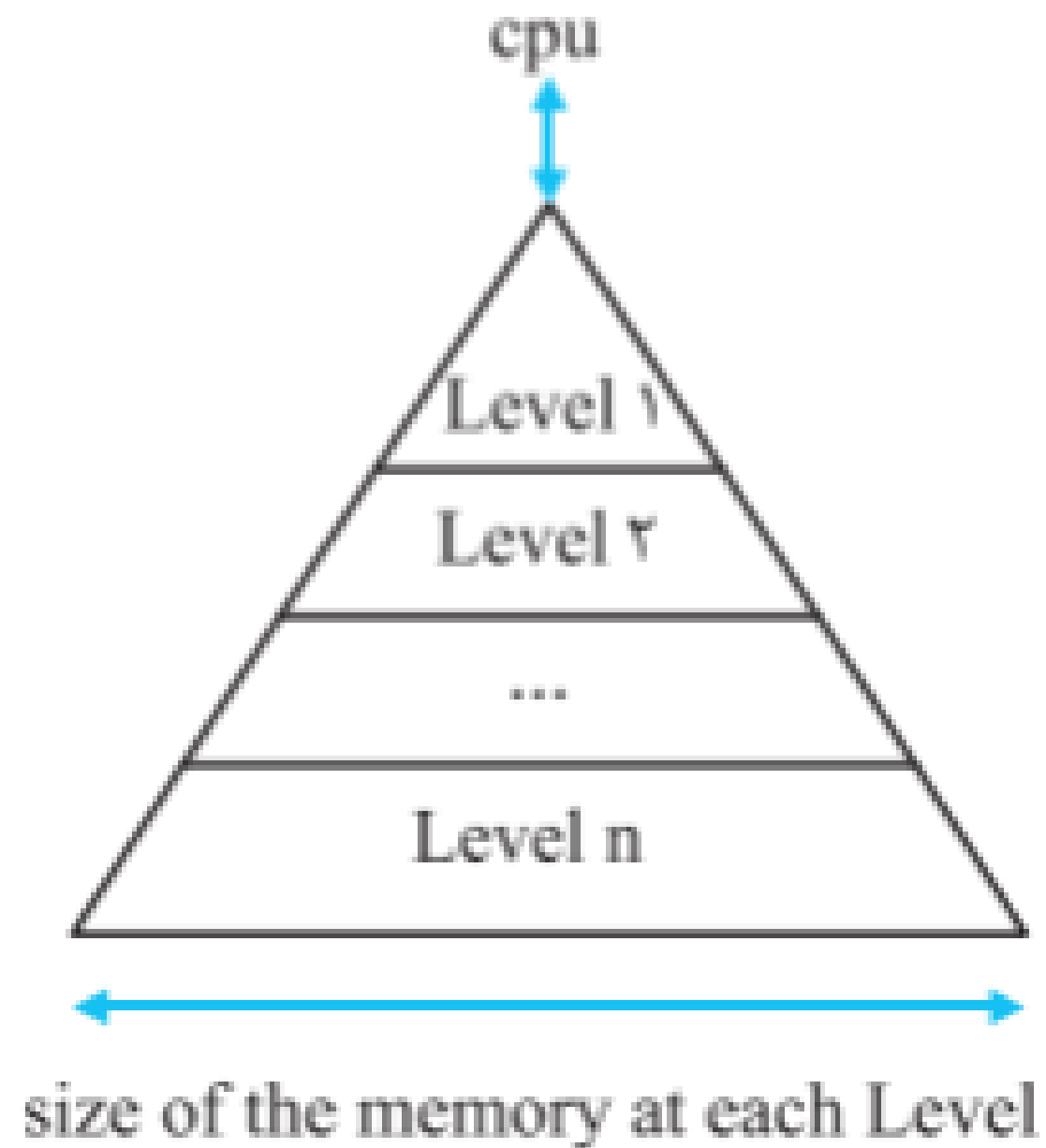


- The main memory is usually **RAM**, but it can also include **ROM**
 - **RAM** is a random access memory (access to all locations takes the same amount of time)
 - RAM is **Read-Write Memory** (RWM), meaning it holds data as long as electrical power is supplied
 - RAM can be built in **two** ways
 - Static RAM (**SRAM**): Bits are stored in flip-flops
 - Dynamic RAM (**DRAM**): Bits are stored in capacitors and require refreshing
 - DRAM consumes less **power** than SRAM but is slower
 - Static RAM can perform better in **speed** compared to DRAM
- **ROM** only has read capability and stores permanent data
 - It's accessed randomly – just like RAM

- The **principle of locality** states that programs access a relatively small portion of their address space at any instant of time.
- **Temporal locality:**
 - Locality in time
 - The locality principle stating that if a data location is referenced, then it will tend to be referenced again soon.
- **Spatial locality:**
 - Locality in space
 - The locality principle stating that if a data location is referenced, data locations with nearby addresses will tend to be referenced soon.

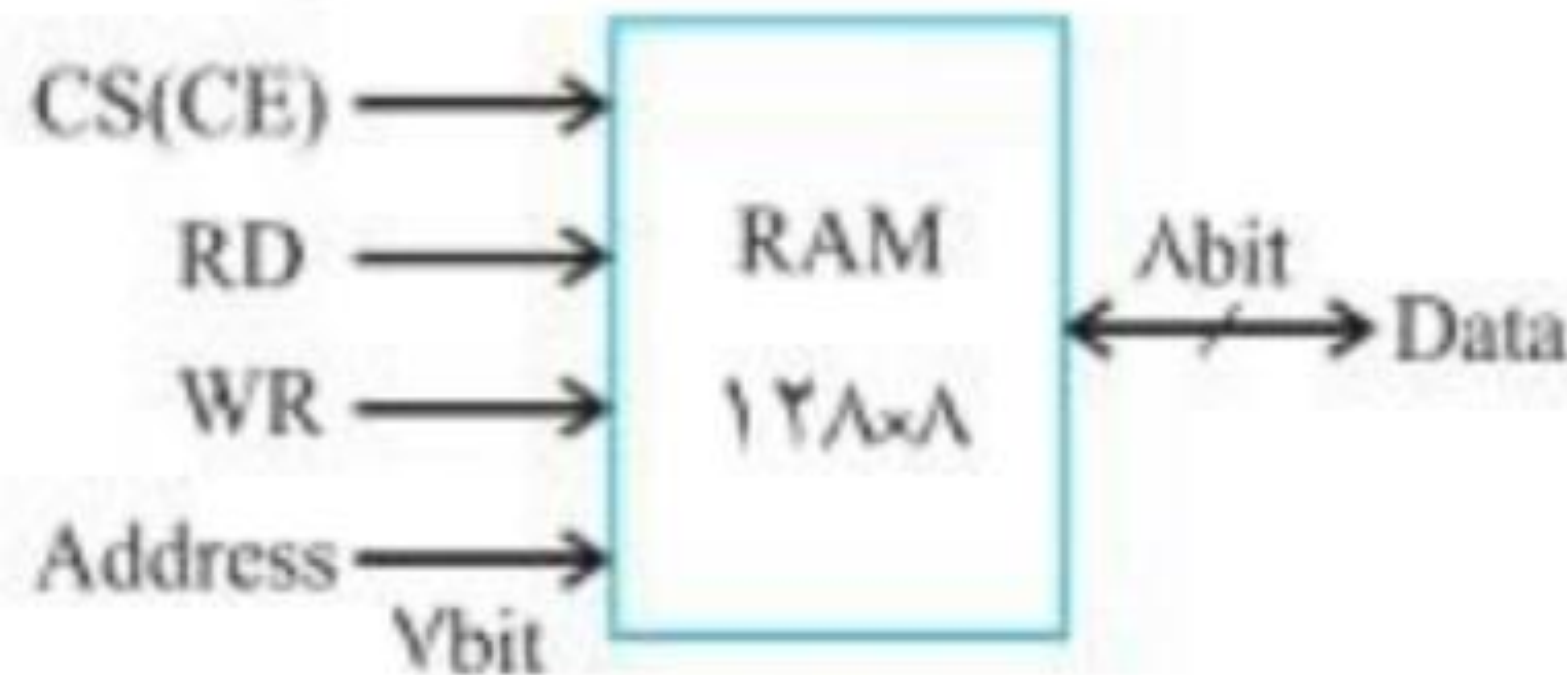
Memory Hierarchy

- A structure that uses multiple levels of memories; as the distance from the processor increases, the size of the memories and the access time both increase.

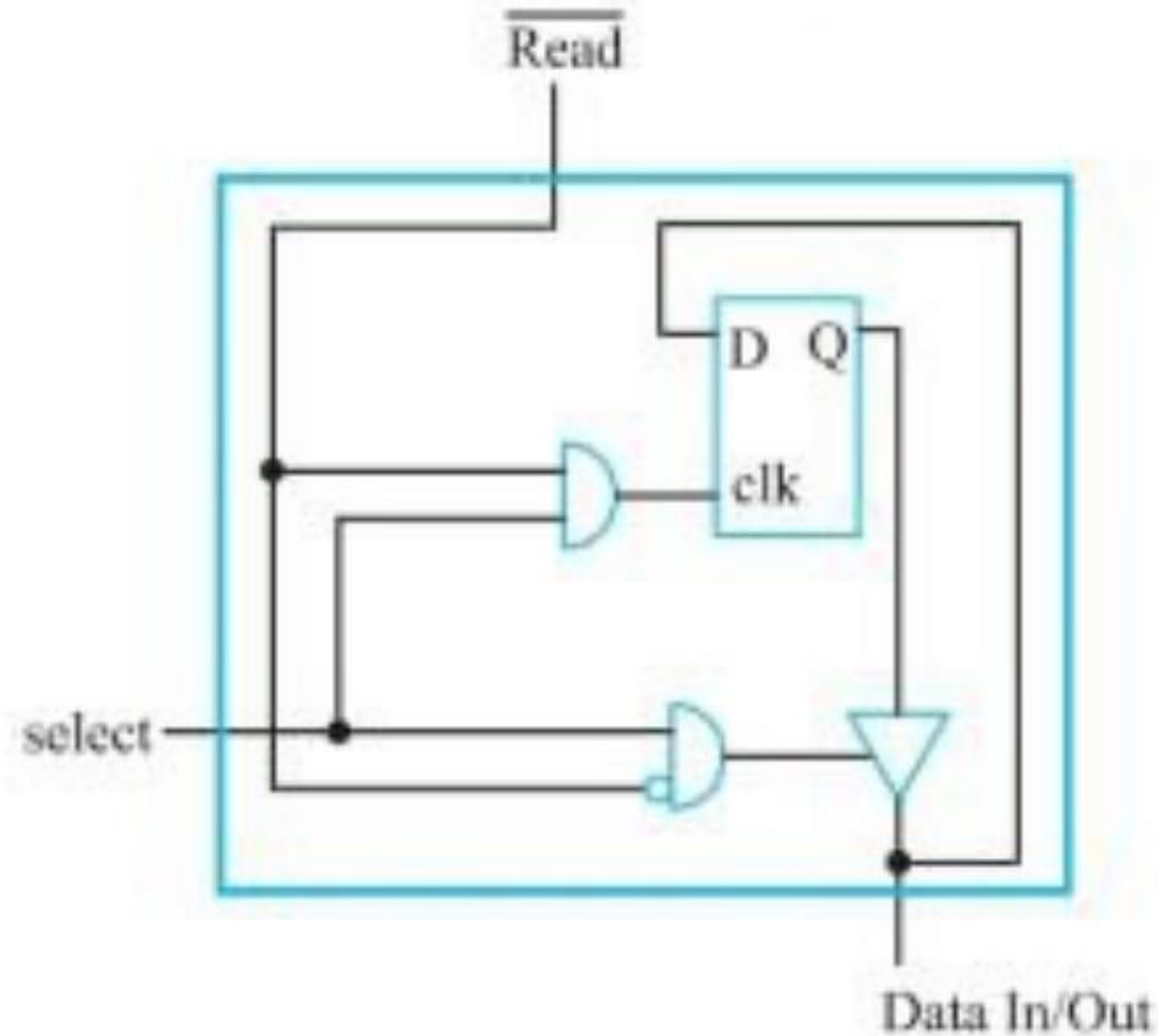


- **The data are similarly hierarchical:** a level closer to the processor is generally a subset of any level further away, and all the data are stored at the lowest level.
- As we move away from the processor, the levels take progressively longer to access.
- A memory hierarchy can consist of multiple levels, but data are copied between only two adjacent levels at a time.
- The upper level—the one closer to the processor—is smaller and faster than the lower level, since the upper level uses technology that is more expensive.

- Access Control to RAM
 - When $CS = 1$, the RAM chip is selected (active)
 - If $RD = 1$, it performs a read operation
 - If $WR = 1$, it performs a write operation
 - If CS is inactive, data lines remain in high impedance state (disconnected)

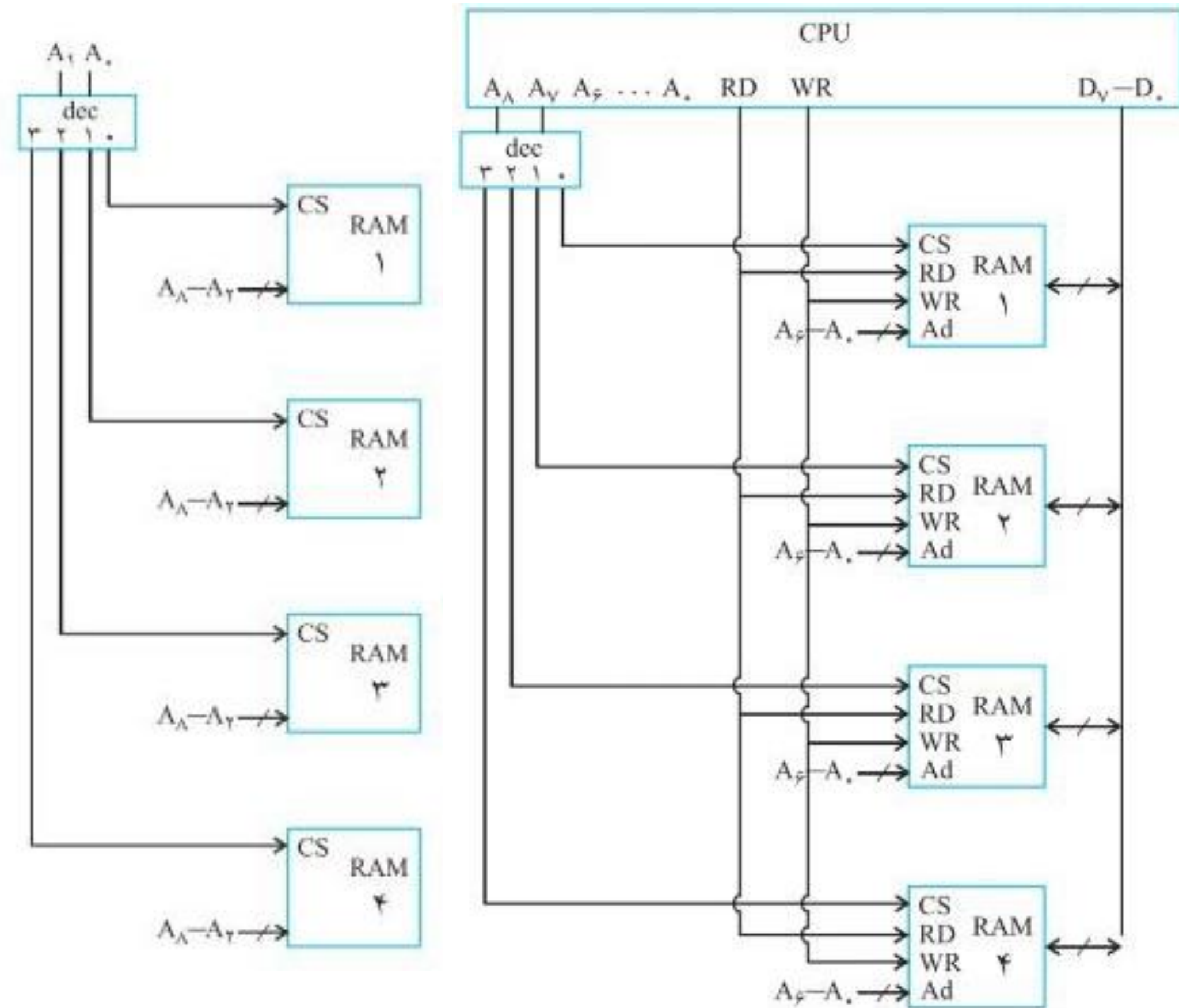


One SRAM Bit Under Telescope



512 * 8 RAM

تراشه	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	قادر ددهی آدرس‌ها
RAM ₁	0	0	x	x	x	x	x	x	x	0-127
RAM ₂	0	1	x	x	x	x	x	x	x	128-255
RAM ₃	1	0	x	x	x	x	x	x	x	256-383
RAM ₄	1	1	x	x	x	x	x	x	x	384-511



- **Interleaving**: Making big memories using smaller chips
- Types of interleaving
 - **High** order interleaving
 - Higher address bits are used to select memory chips
 - **Low** Order Interleaving
 - Lower address bits are used to select memory chips
 - **Helps processors access consecutive addresses efficiently, improving speed**
- **Note**: We can also use MUXes in order to interleave memories

Example

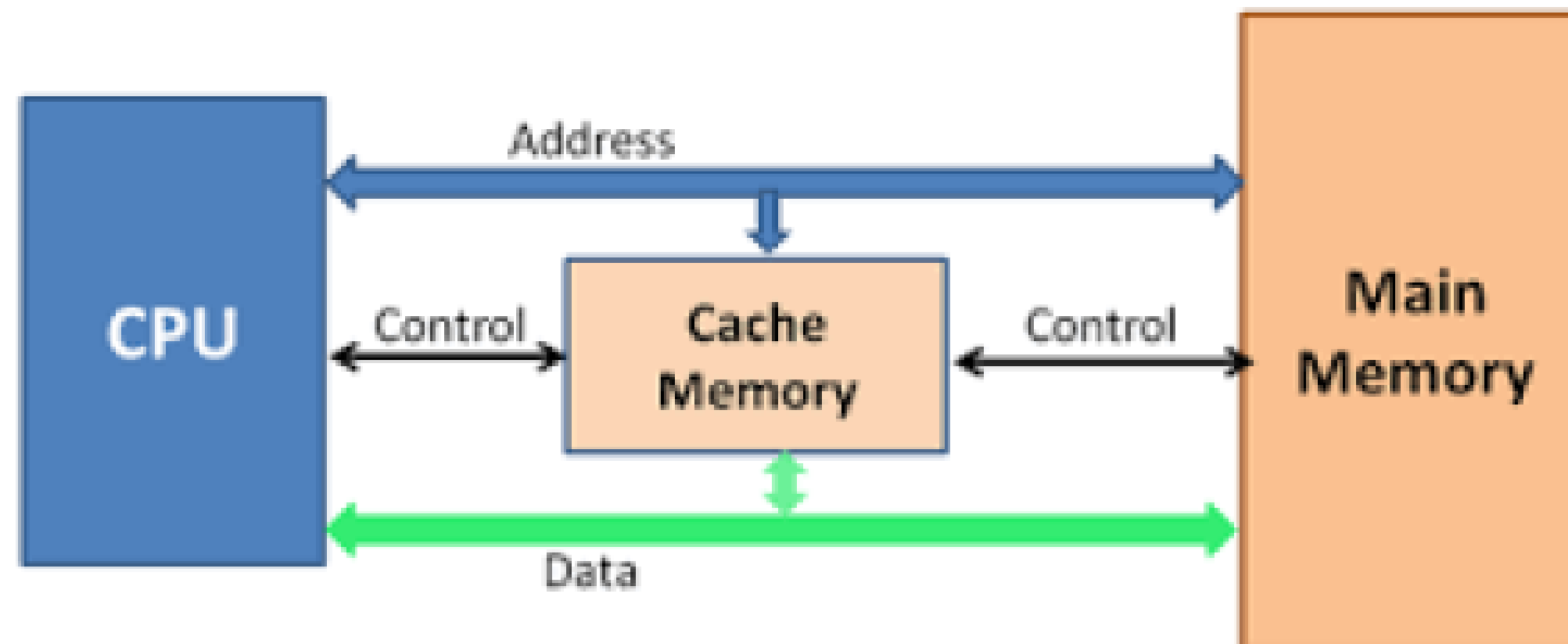
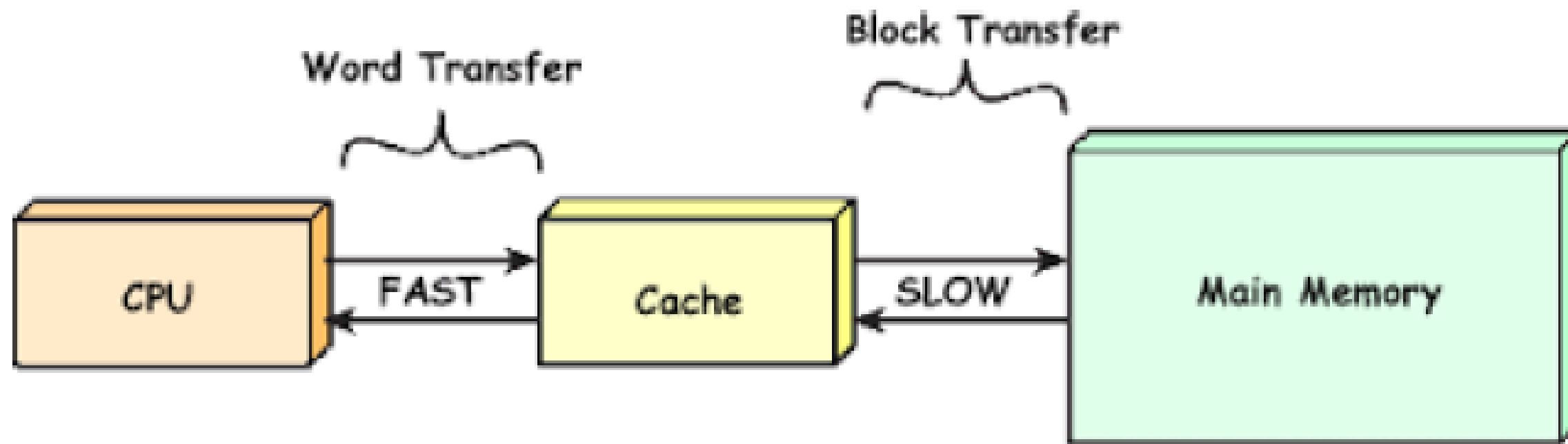
- If we want to store an 8×8 matrix in memory, and if the memory consists of 8×8 KB chips, in what way should we interleave these memories in order to increase access speed to the elements of the second column in matrix?

- Answer

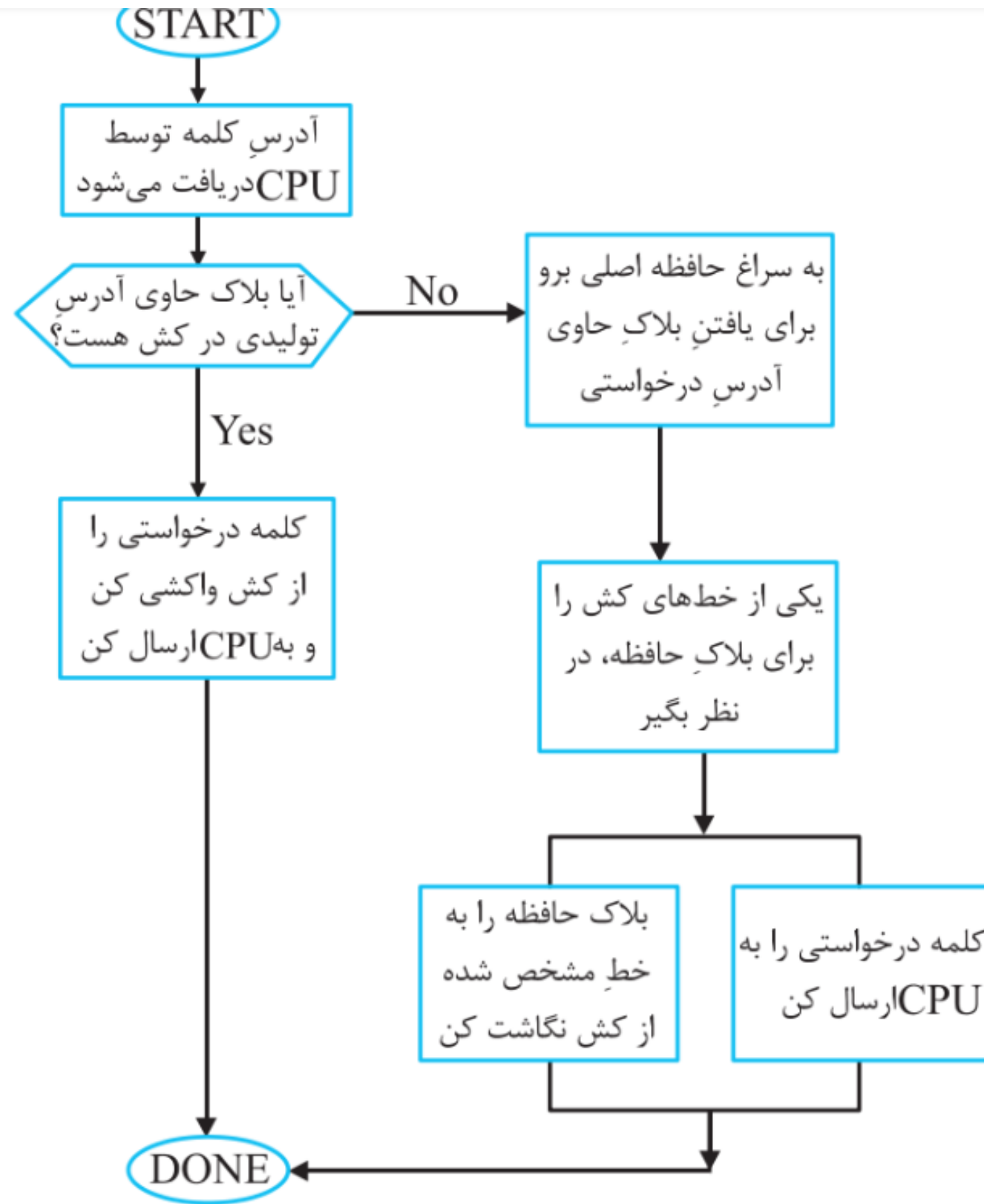
$$A = \begin{bmatrix} a_{11} & a_{12} & a_{18} \\ a_{21} & a_{22} & a_{28} \\ \vdots & \vdots & \vdots \\ a_{81} & a_{82} & a_{88} \end{bmatrix} \Rightarrow$$

عنصر	آدرس	A_{15}	...	A_6	A_5	A_4	A_3	A_2	A_1	A_0
a_{12}	1	0	...	0	0	0	0	0	0	1
a_{22}	9	0	...	0	0	0	1	0	0	1
a_{32}	17	0	...	0	0	1	0	0	0	1
$\vdots$										

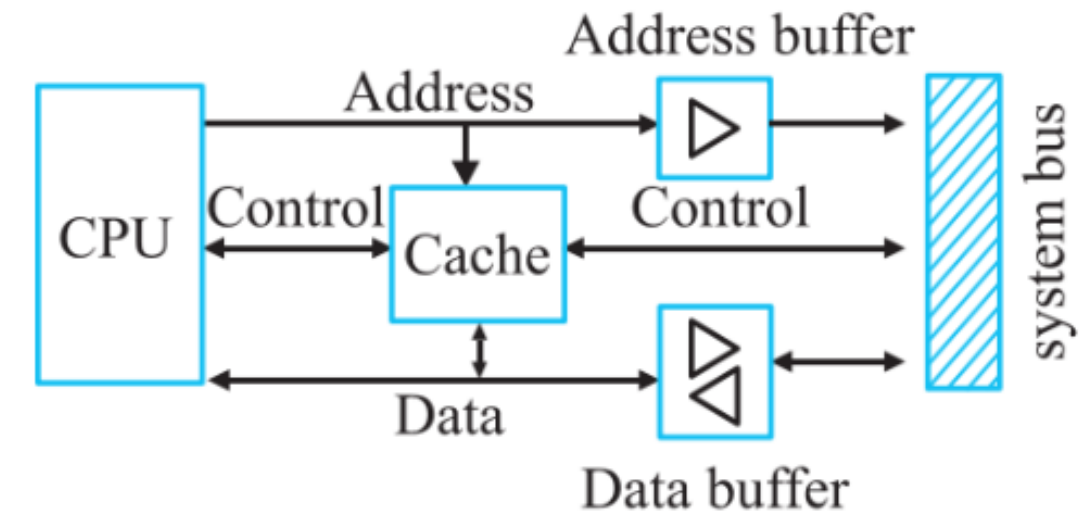
Memory Hierarchy



- A small and fast memory which is between CPU and main memory.
- Contains copy of some of the main memory cells
- CPU's memory access is first checked with the contents of cache.
 - If in cache, then hit.
 - If not in cache, then miss:
 - The block containing the data is transferred to cache
 - Then the data is transferred to CPU from cache
- There can be multiple layers of cache



(a) عملیات خواندن از کش



(b) ساختار موازی کش/حافظه که اصطلاحاً Load through نیز می‌گویند.

■ Block (Line)

- the minimum unit of information that can be either present or not present in the two-level hierarchy.
- Usually, we transfer an entire block when we copy something between levels.

■ Hit

- If the data requested by the processor appear in some block in the upper level, this is called a hit.

■ Miss

- If the data are not found in the upper level, the request is called a miss.
- The lower level in the hierarchy is then accessed to retrieve the block containing the requested data.

■ Hit rate (hit ratio)

- The fraction of memory accesses found in a level of the memory hierarchy.
- It is often used as a measure of the performance of the memory hierarchy.

■ Miss rate ($1 - \text{hit rate}$)

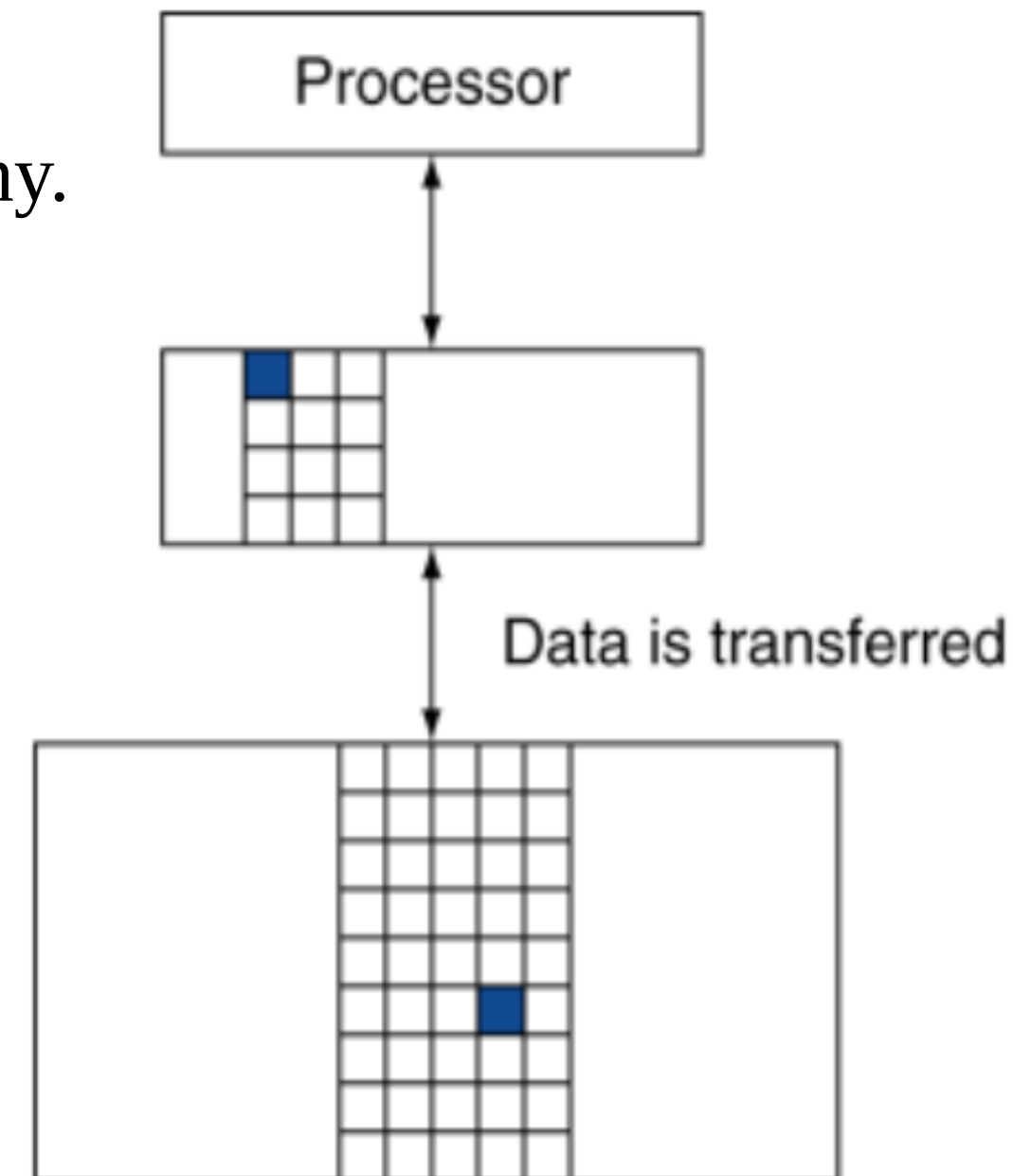
- The fraction of memory accesses not found in a level of the memory hierarchy.

■ Hit time

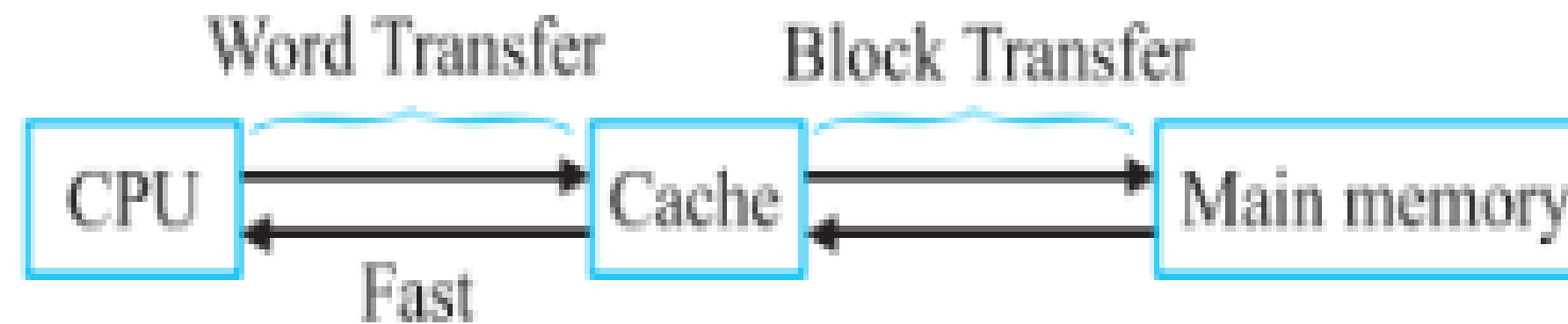
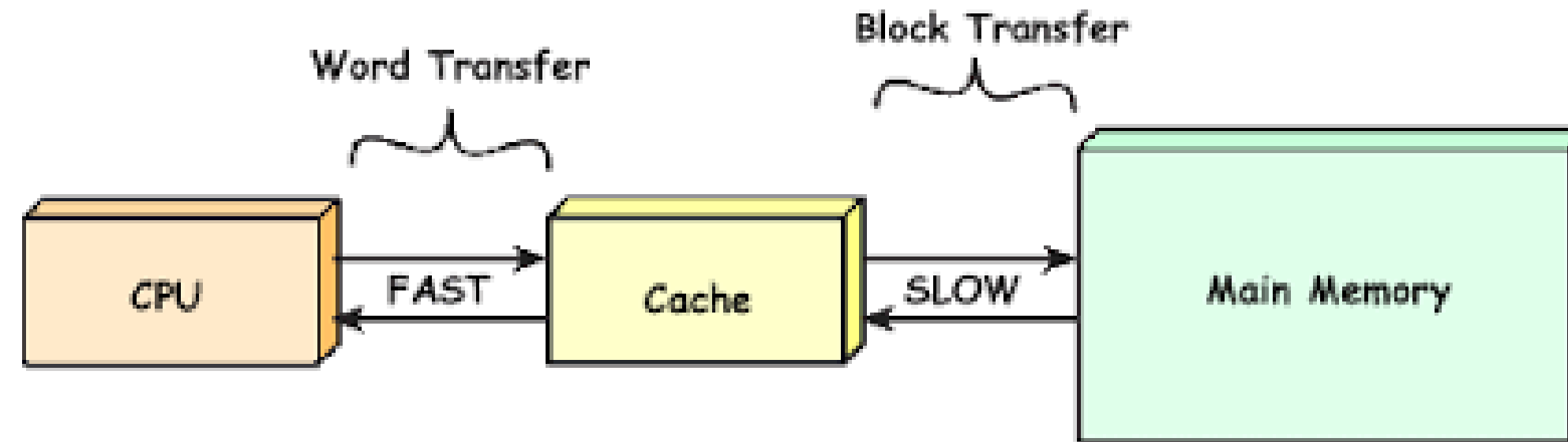
- The time required to access a level of the memory hierarchy, including the time needed to determine whether the access is a hit or a miss.

■ Miss penalty

- The time required to fetch a block into a level of the memory hierarchy from the lower level

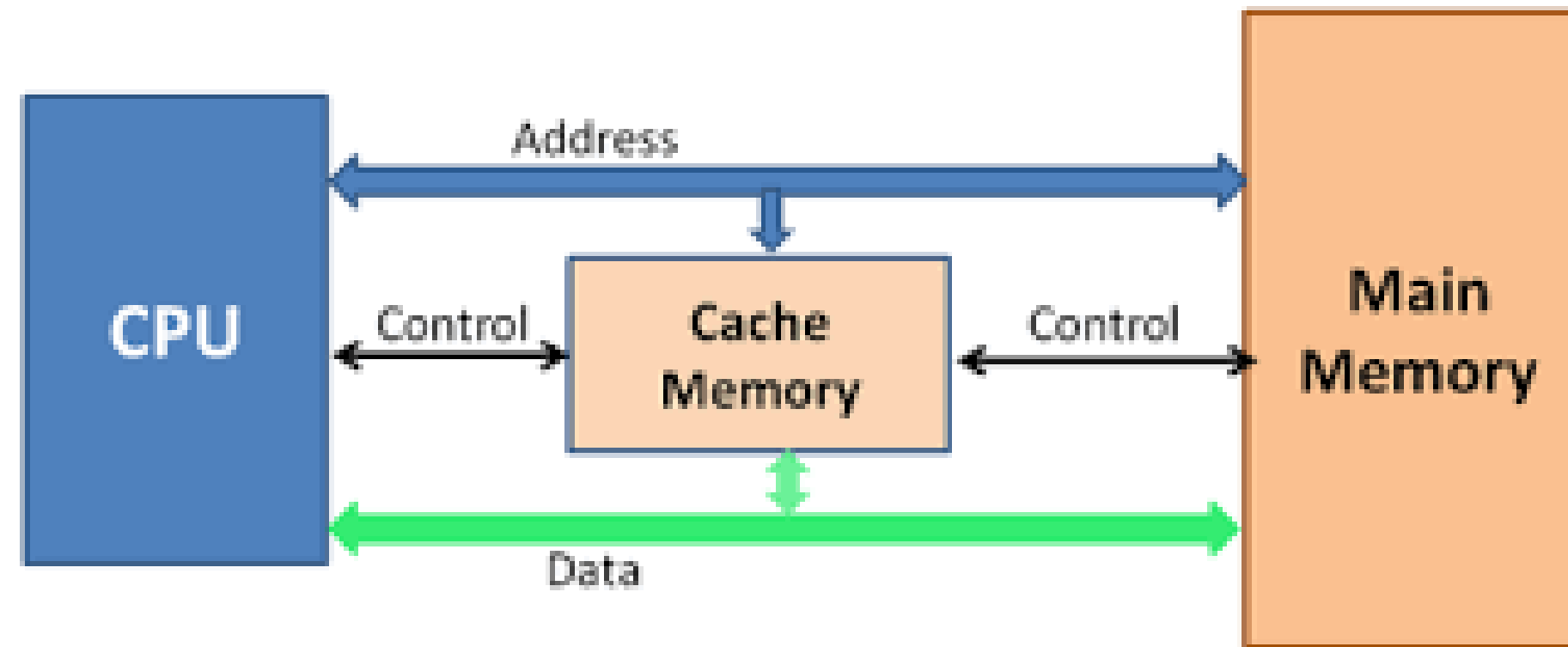


Average Access Time (Series)



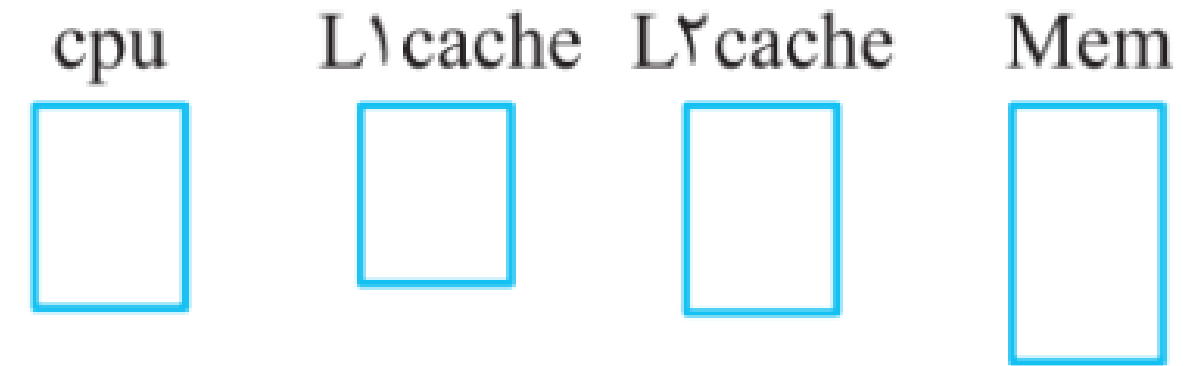
$$t_{ave} = h.t_c + (1-h).(t_m + t_c) = t_c + (1-h).t_m$$

■ Load Through (Parallelism):



$$t_{ave} = h.t_c + (1-h).t_m$$

- For 3 levels of cache:



$$t_{ave} = t_1 + (1 - h_1)t_2 + (1 - h_1)(1 - h_2)t_3$$

- For 3 levels of cache and Parallelism (load through) :

$$t_{ave} = h_1 t_1 + (1 - h_1)h_2 t_2 + (1 - h_1)(1 - h_2)t_3$$

- When miss occurs, a block from the main memory is transferred to a line in cache, this is called cache mapping.
- Three methods of cache mapping:
 1. **Direct mapping**
 2. **Associative mapping**
 3. **Set-associative mapping**

Direct Mapping

- In this method, there is a specific line in cache for each block in main memory.

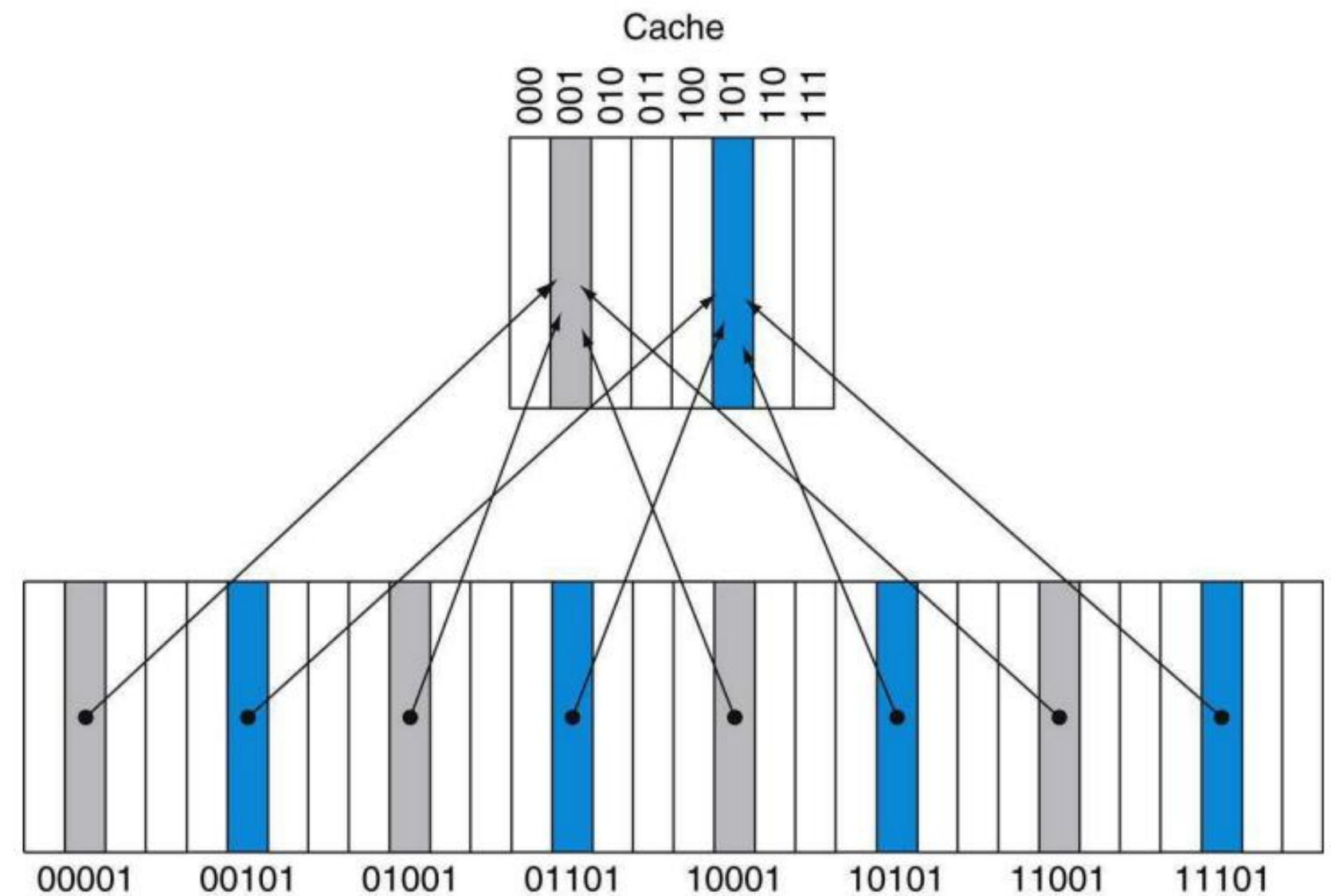
- Mapping is done using this formula:

$$i = j \bmod m$$

i = index of cache line

m = number of cache lines

j = index of main memory block



- If m is a power of 2 ($m = 2^x$) then i is the x right bits of i (no calculation is needed)

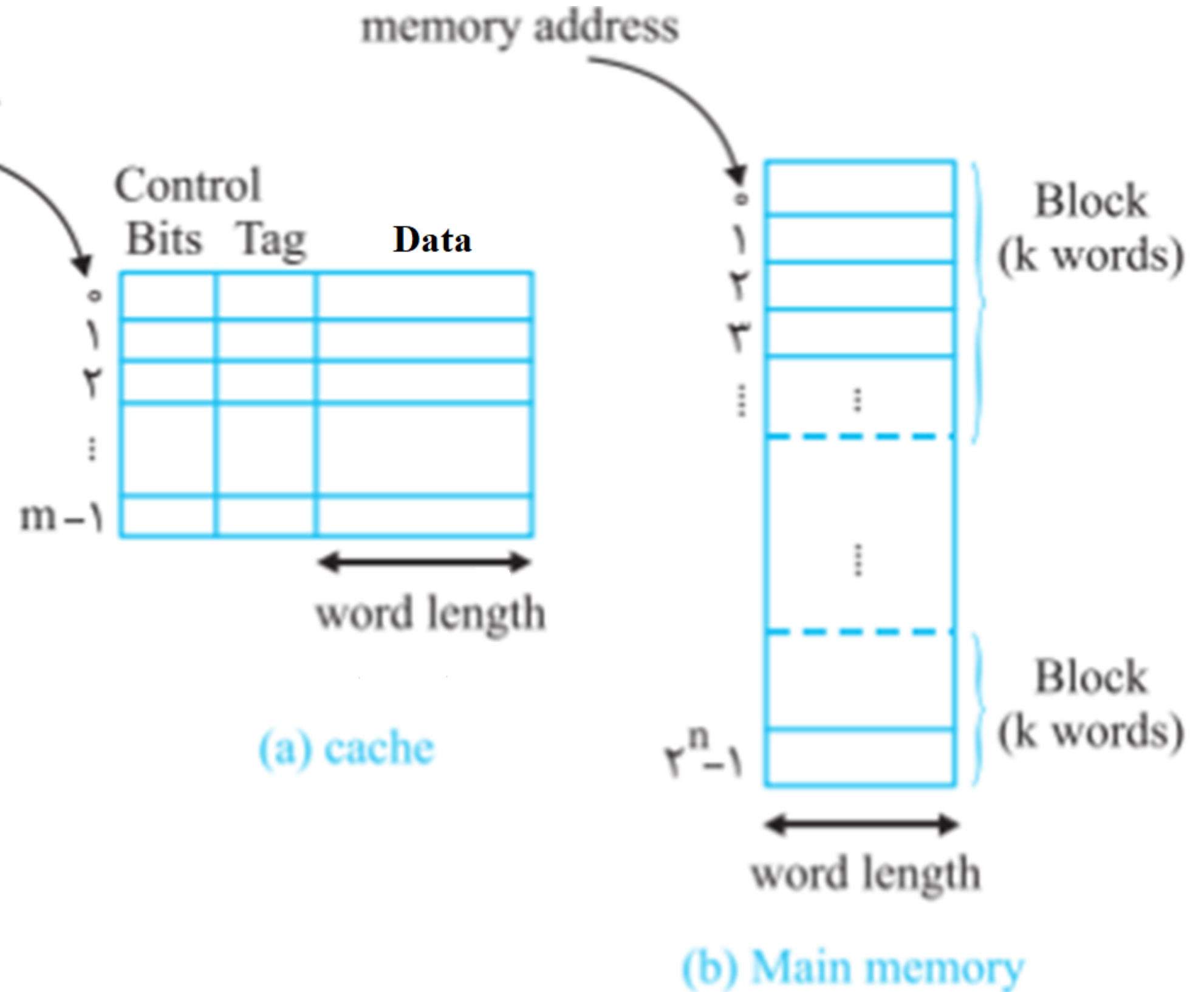
Direct Mapping

■ Tag

- A field in a table used for a memory hierarchy that contains the address information required to identify whether the associated block in the hierarchy corresponds to a requested word.

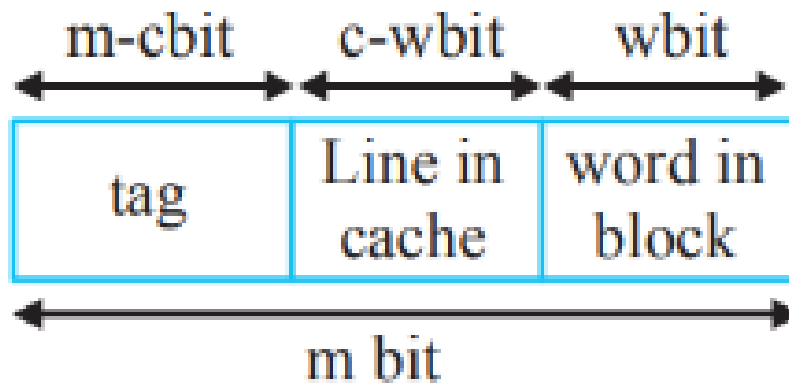
■ Valid bit

- A field in the tables of a memory hierarchy that indicates that the associated block in the hierarchy contains valid data.



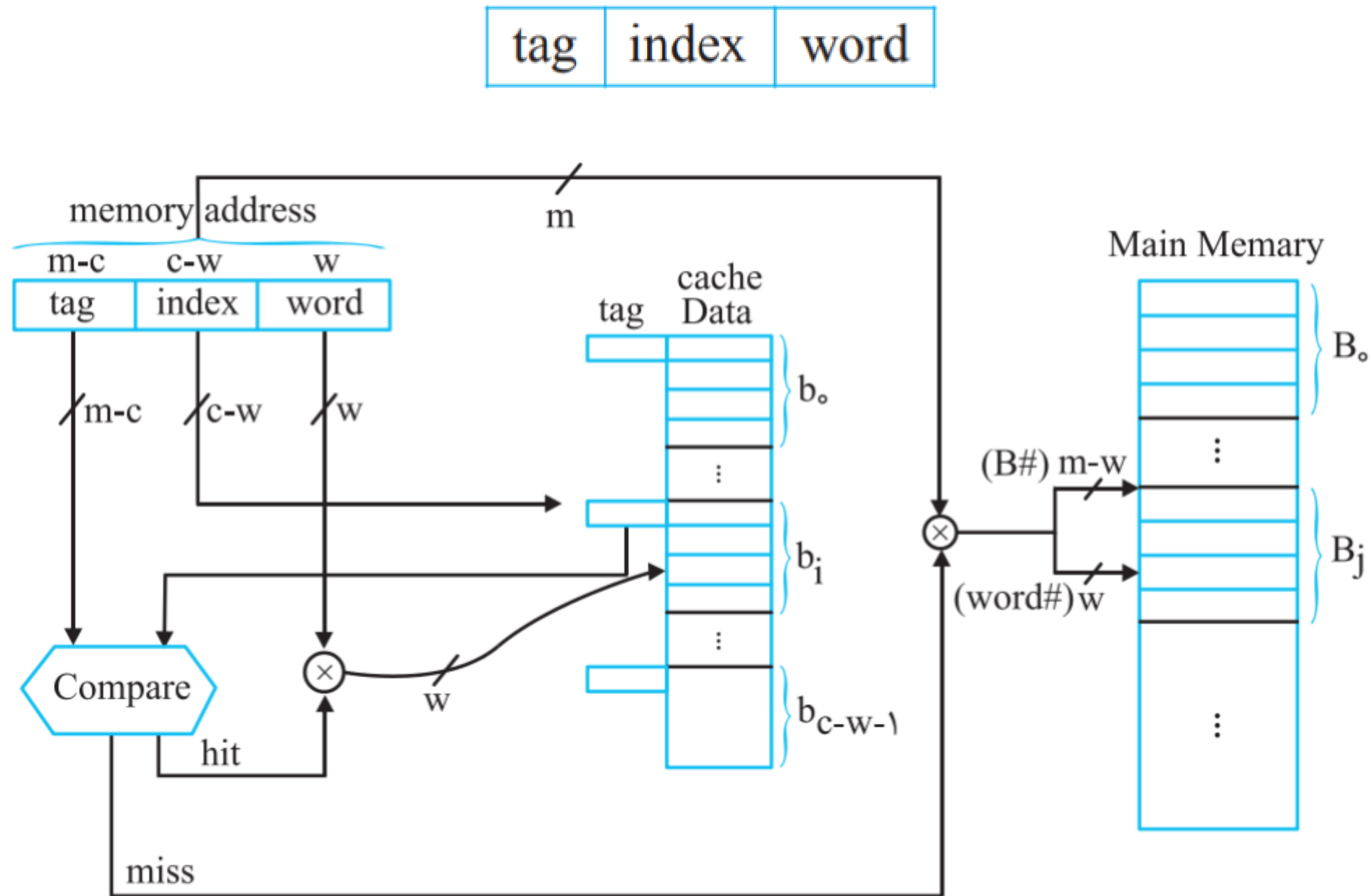
Direct Mapping

- Number of main memory blocks = $\frac{2^m}{2^w} = 2^{m-w}$
- Number of cache lines = $\frac{2^c}{2^w} = 2^{c-w}$
- Main memory to Cache size ratio = $\frac{2^m}{2^c} = 2^{m-c}$
- Length of address sent to the main memory = m bit

- Format of address sent to the main memory = 

- Total bits of cache = $(2^c \times \text{wordsize} + 2^{c-w} \times (m - c) + 2^{c-w} \times 1) \text{ bit}$

Direct Mapping



- **Example:** a direct cache has 64 16-byte blocks. Address 1200 of the main memory is going to be mapped to which block of the cache? By referencing address 1200, which addresses will be transferred to cache?

Example: Main memory has 2G words. The size of cache is 1M words and each block is 256 words. The addressable unit is a word. Indicate the format of the address.

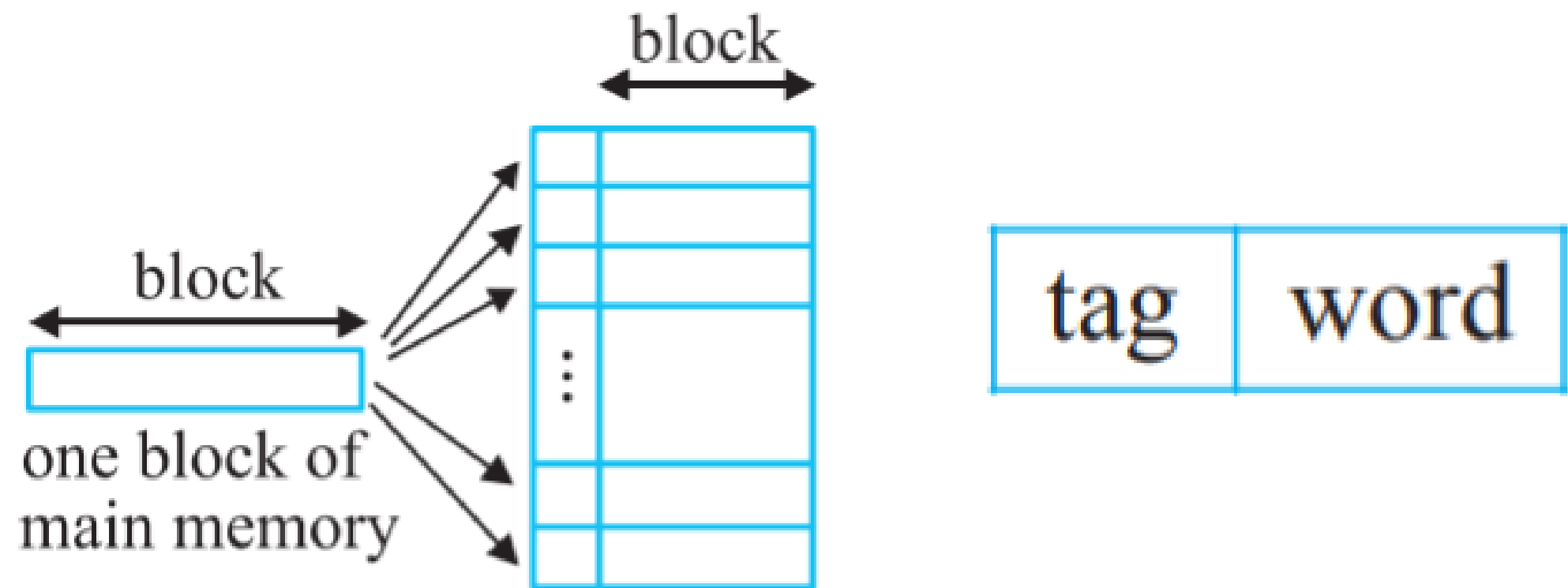
Example: A direct cache consists of 8 blocks of 4 words and the processor accesses addresses 0 to 63 ten times. The cache is initially empty. Calculate the hit rate?

In the previous example, the cache contains 4 blocks of 8 words, what is the hit rate?

In the previous example, the cache contains 8 blocks of 8 words, what is the hit rate?

Associative mapping

- In this method, any block of the main memory can be mapped to any line of cache
- The associative memory stores both the address and data of the memory word.
- This allows any location in cache to store any word from main memory.
- The decision to which lines to replace is determined by the **replacement policy**
- **Famous Replacement Policies:**
 - LRU (usually most hit rate)
 - FIFO
 - LFU
 - random



Replacement Policies

- **LRU** – Least Recently Used (usually most hit rate)
 - Chooses the block that hasn't been access for a longer time
 - A separate list is needed to keep track of the recently used which puts the most recent at the top of the list
- **FIFO** – First In, First Out
 - chooses the block that has been in the cache the longest
 - Needs a counter to keep track
- **LFU** – Least Frequently Used
 - chooses the block that has had the least accesses.
 - Needs a counter for every line of cache
- **Random**
 - Chooses randomly

- Assume main memory is 64x8, each block is 4 bytes, and cache is 4 blocks. What is the addressing system? If address 5 is generated, which block of the cache will it be moved to?

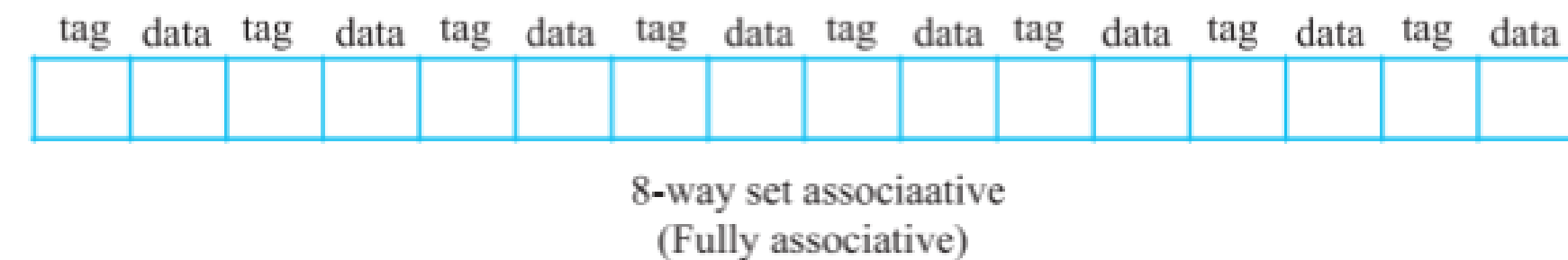
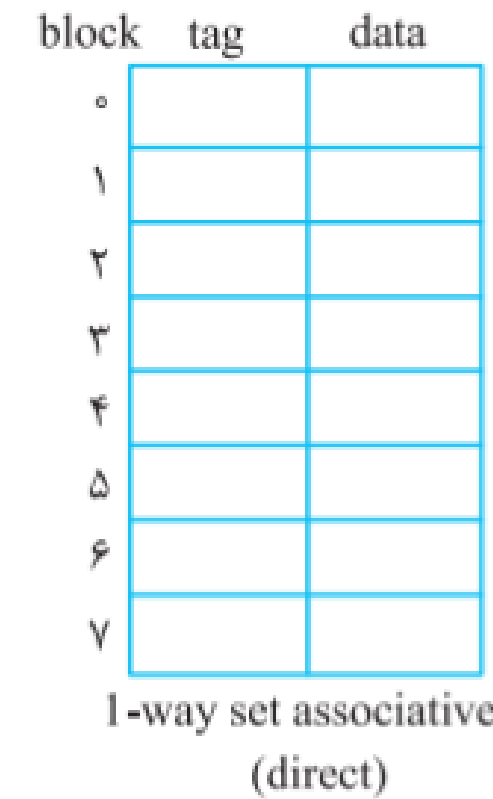
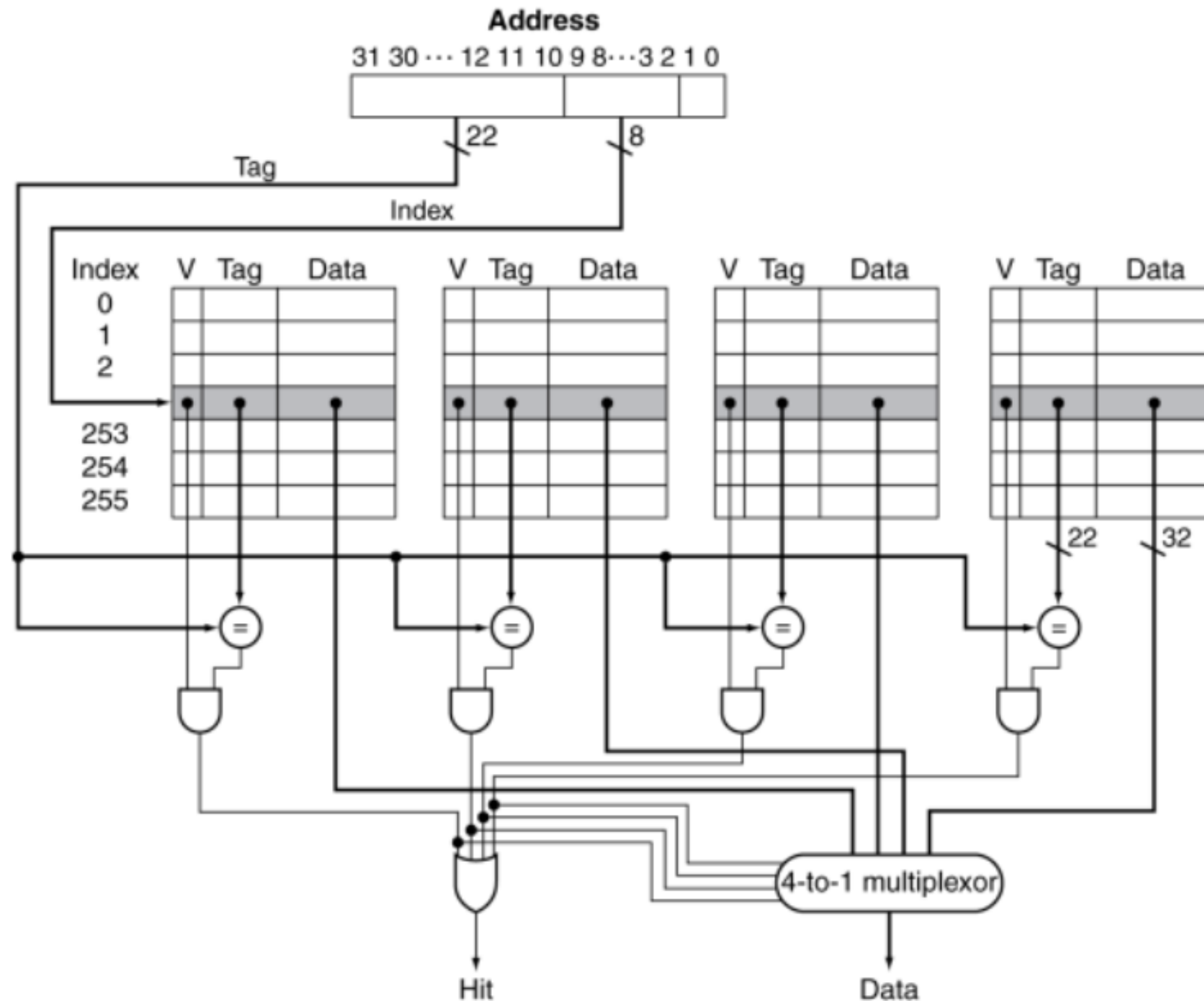
Set-Associative mapping

- a compromise between direct-mapped and fully-associative mapping in cache systems
- n-way set associative:
 - Each set contains n entries.
 - Block number determines which set the requested item is located in.
 - Search all entries in a given set at once.
 - n comparators (less expensive).

[illegible]

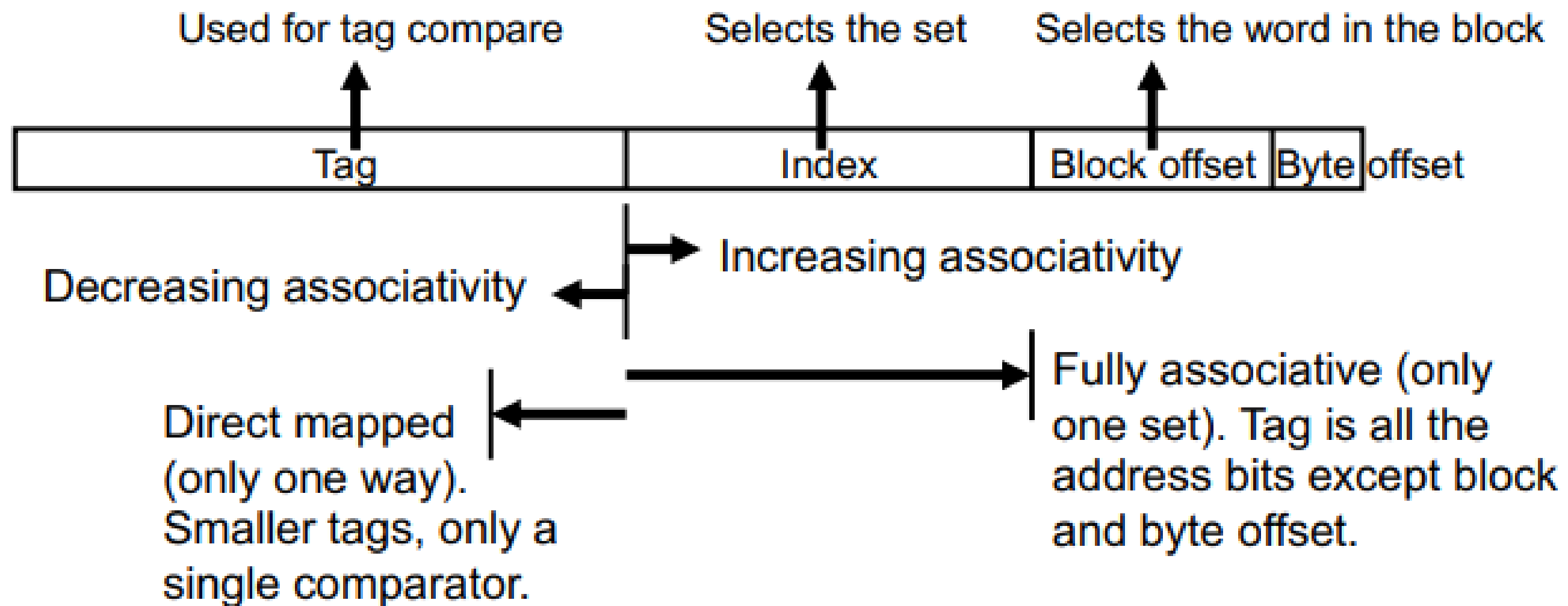
Figure 12-15 Two-way set-associative mapping cache.

4-way Set-Associative mapping



Range of Set Associative Caches

- For a fixed size cache, each increase by a factor of two in associativity doubles the number of blocks per set (i.e., the number or ways) and halves the number of sets – decreases the size of the index by 1 bit and increases the size of the tag by 1 bit.



- **Example:** main memory is 64 bytes. Cache has 32 bytes, and each block is 4 bytes. Addressable unit is a byte. Show the cache in a 2-way set associative setting. Show the mapping between cache and main memory.

Example: Example: Suppose a processor generates addresses 0, 1, 2, ..., 8 three times in a row. If we have a cache with a capacity of 8 words, what is the hit rate in each of the following cases?

1. Direct-mapped cache
2. Fully associative cache with FIFO replacement policy
3. Set associative cache with 2 blocks per set with FIFO replacement policy

Example: Example: Suppose a processor generates addresses 0, 1, 2, ..., 8 three times in a row. If we have a cache with a capacity of 8 words, what is the hit rate in each of the following cases?

1. Direct-mapped cache

$$\text{Hit Rate} = \frac{2 \times 7}{3 \times 9} = \frac{14}{27}$$

2. Fully associative cache with FIFO replacement policy

0

1. Set associative cache with 2 blocks per set with FIFO replacement policy

00	M[0]	M[4]
01	M[1]	M[5]
10	M[2]	M[6]
11	M[3]	M[7]

0	1	2	3	4	5	6	7	8
M	M	M	M	M	M	M	M	M
0	1	2	3	4	5	6	7	8
M	H	H	H	M	H	H	H	M
0	1	2	3	4	5	6	7	8
M	H	H	H	M	H	H	H	M

$$\text{Hit Rate} = \frac{2 \times 6}{3 \times 9} = \frac{4}{9}$$

Example: Suppose a matrix of size 4×8 is used, where each element is a word. The matrix is saved in addresses from 1000 to 1031 column-wise in the main memory. The cache contains 8 blocks, each 2 words. , the processor accesses. Find the hit rate in direct, associative, and 2-way set associative mapping. In each method, what percentage of cache is used? (use LRU as the replacement policy)

■ CPU Write Incoherency

- If we only write to cache, the same variable will have a different and invalid value in main memory
- What could go wrong?

■ Write-Through Cache

- Update main memory ALONGSIDE cache on every write request
- Performance Issues
- Cache miss
 - If our variable (whether it has a single or multiple blocks) isn't in cache AT ALL, we won't need to write to cache
 - If our variable consists of multiple memory blocks and some of them aren't in the cache, we OUGHT TO move the whole variable from memory to cache first and then write to cache

■ **An Improvement: Write Buffer**

- Write requests are stored in a buffer
- The buffer is taken care of every now and then (Or when it's full)

■ **Another Solution: Write-Back**

- We only write to cache
- We'll write cache data to main memory when we want to replace the corresponding block
- **Modify (Dirty)** bit for each cache block
 - Indicates whether the block has been changed since it entered the cache
- Harder than **Write-Through**
 - **DMA**
 - Uses less power and bandwidth

■ Write Allocate

- On a write miss, the block is transferred to the cache

■ No Write Allocate

- On a write miss, the block is not transferred to the cache
- Write happens only in main memory
- Suitable for full write-through policies

■ Example

درخواست	write allocate	no write allocate
write m[۱۰۰]	miss	miss
write m[۱۰۰]	hit	miss
read m[۲۰۰]	miss	miss
write m[۲۰۰]	hit	hit
write m[۱۰۰]	hit	miss

■ Write-Through Policy with Write Allocate

- If write allocate is used, average memory access time will be

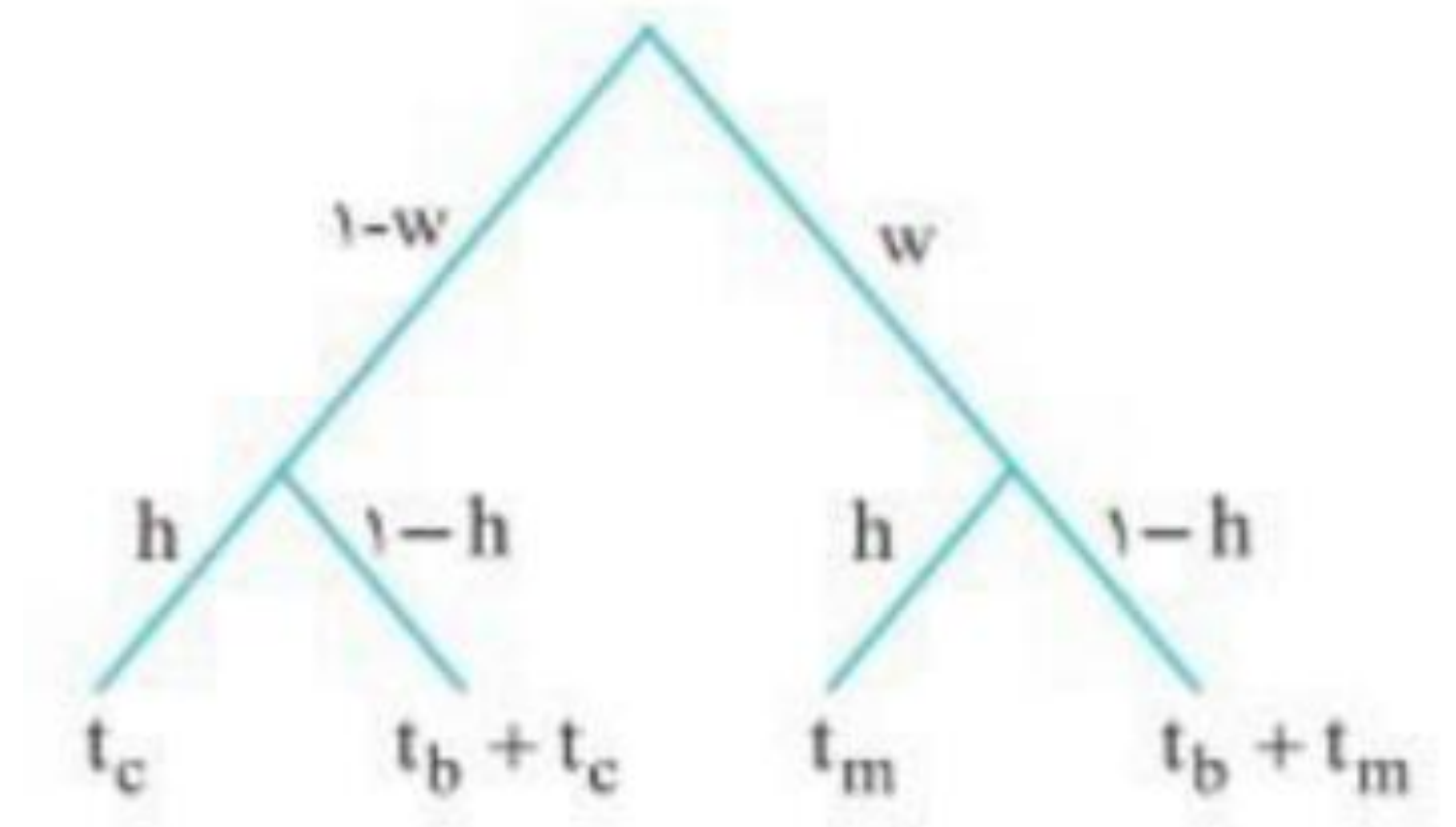
- $t_{avg} = t_c + (1 - h)t_b + w(t_m - t_c)$

- Where

- t_b = time to transfer a block
 - w = percentage of accesses that are **writes**

■ Explanation: Probability tree

- Parallel read and write model
- If miss happens ((1-h) probability)
 - Block fetched from main memory (t_b)
 - Word is written either in cache or main memory depending on policy

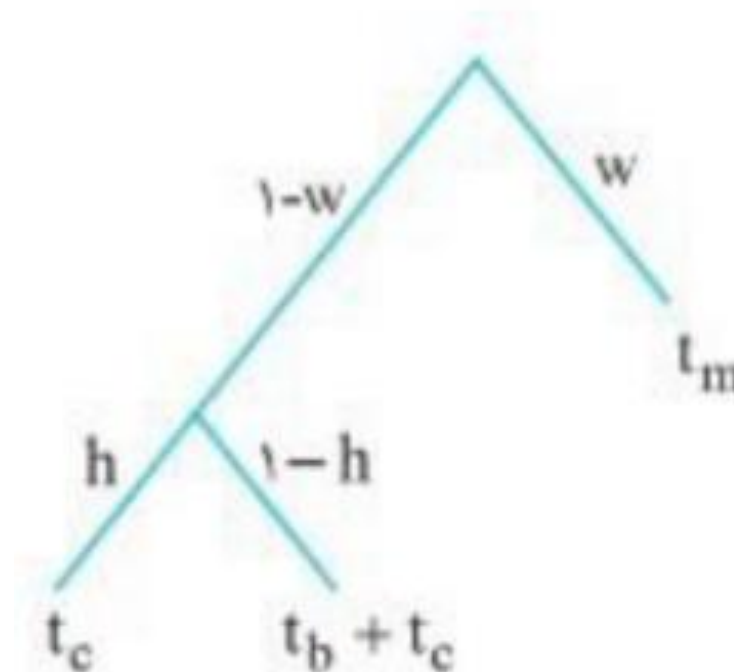


Average Memory Access Time

■ No Write Allocate Policy

- Average memory access time

- $t_{ave} = t_c + (1 - w)(1 - h)t_b + w(t_m - t_c)$
- Look at this graph =>

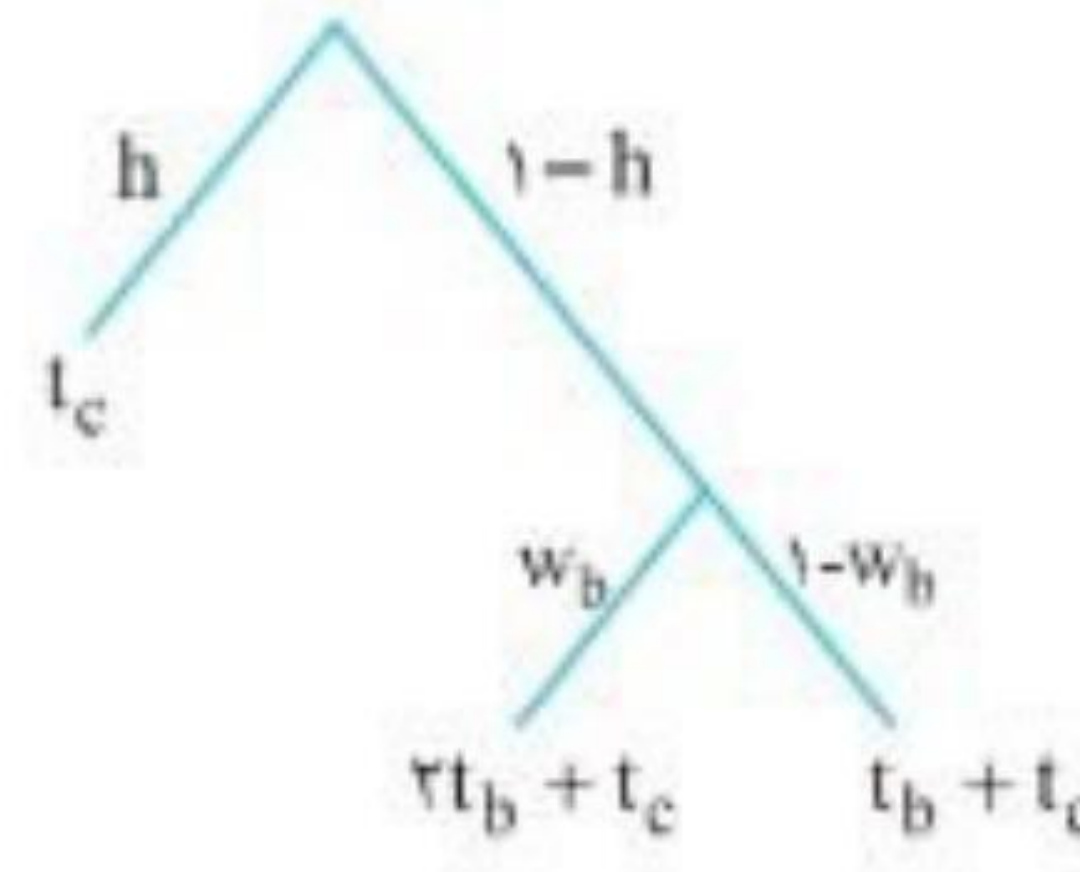


■ Write-Back Policy

- Typically uses write allocate method

- Average access time

- $t_{ave} = t_c + (1 - h)t_b + w_b(1 - h)t_b$
- Look at the following graph



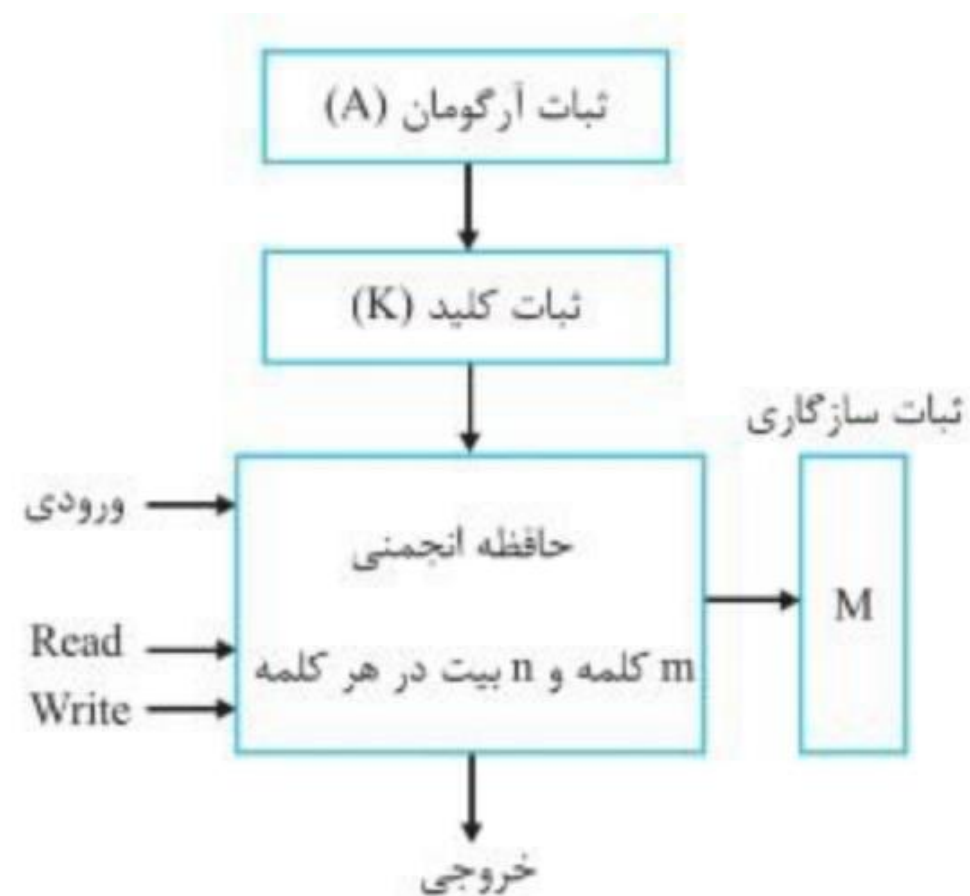
- **CPU time is divided into**
 - Clock cycles for program execution
 - Clock cycles waiting for memory access
- CPU time formula
 - **CPU time** = (CPU execution clock cycles + memory stall clock cycles) × clock cycle time
- **Memory stall clock cycles** = read stall cycles + write stall cycles
- **Read stall cycles** = (reads / program) × read miss rate × read miss penalty
- For Write-Through policy
 - **Write stall cycles** = (writes / program) × write miss rate × write miss penalty + write buffer stalls
- If write buffer is large enough, write buffer stalls can be ignored. Then:
 - **Memory stall clock cycles** = (memory accesses / program) × miss rate × miss penalty
 - OR: **Memory stall clock cycles** = (instructions / program) × (misses / instruction) × miss penalty

Associative Memory

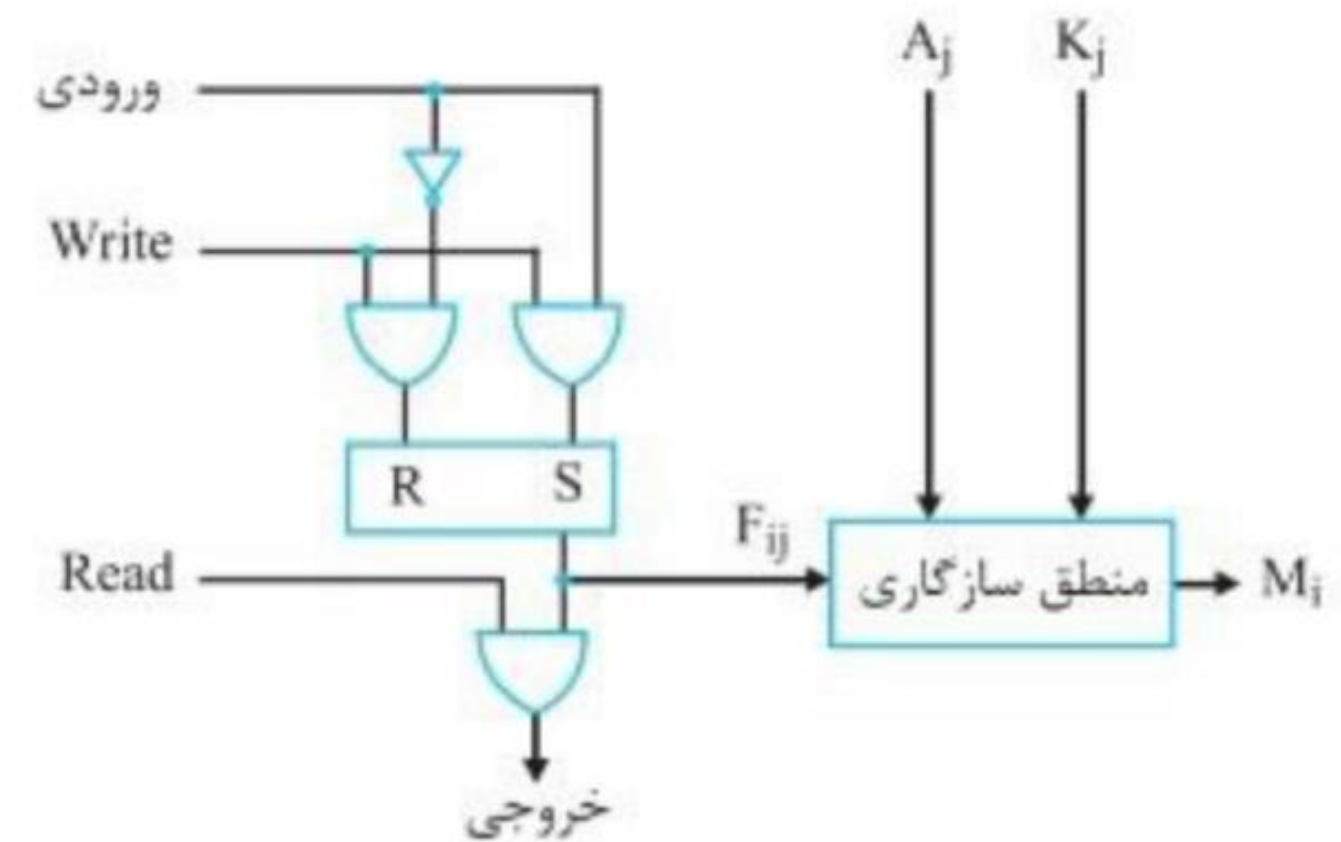
- A memory structure designed to store data so that future searches can be performed **based on content** rather than addresses
 - “See, I don’t know where she lives, but I could draw her from memory like it’s a police sketch”
- Does not require an address to read/write data
 - Writes data into the closest accessible row
 - Has logical comparison circuits to match input data against all stored entries
- The word that’s supposed to be compared to each memory word is stored in **Argument Register**
- CAM can be masked to match partial contents using mask bits
 - Meaning only masked bits are compared
- Also called **Content Addressable Memory (CAM)**

■ Capabilities of Associative Memory

- Reading a word from memory
- Writing a word into memory
- Finding a word(s) that contains a specific data/pattern
- Writing in a word(s) that has a specific data/pattern **simultaneously**



شکل (۱۸۸) شمای کلی حافظه CAM.

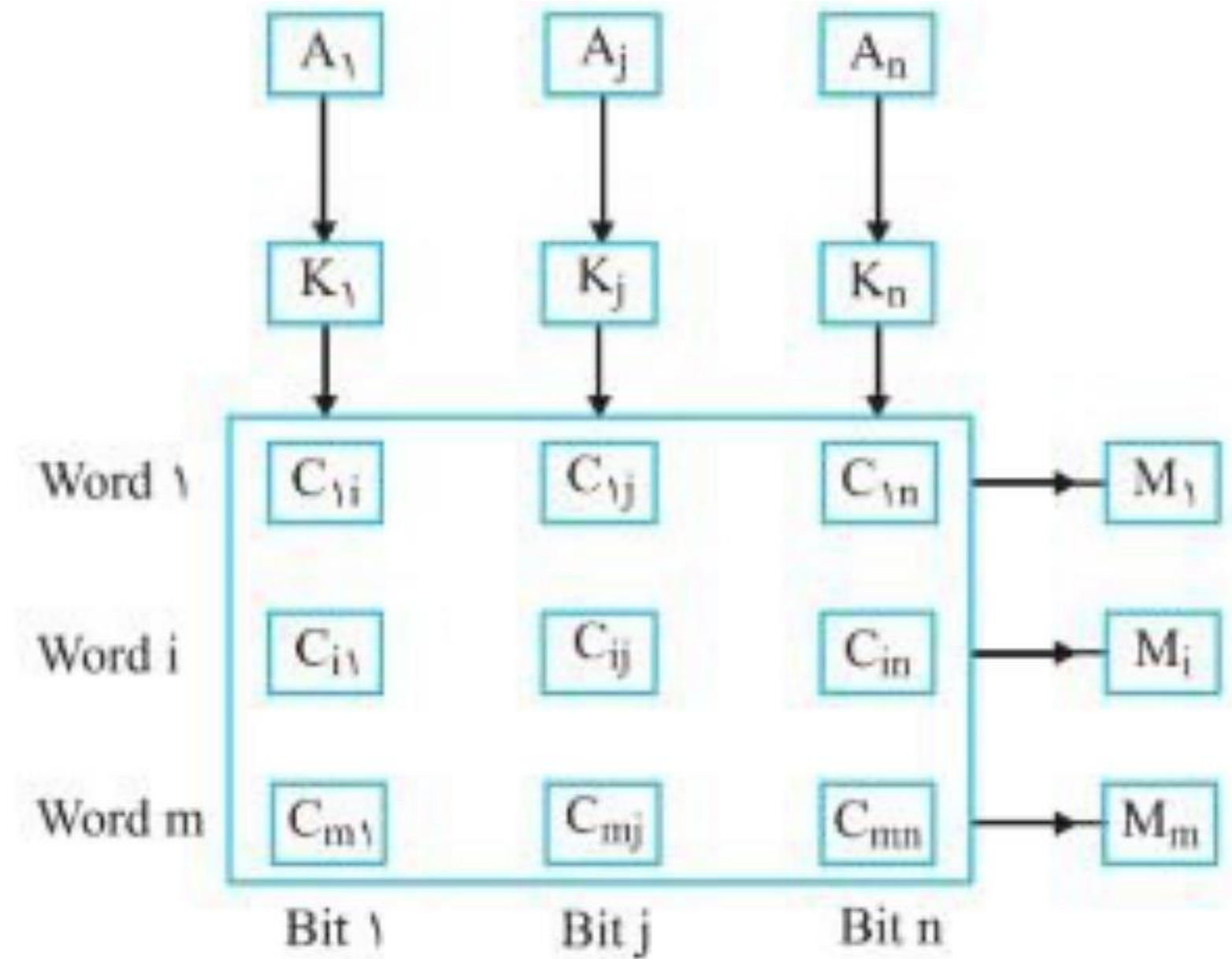


■ Associative Memory Logic

- A: Argument Register
- K: Key Register
- C: CAM cell
- M: Match Register

$$M_i = (x_1 + k'_1)(x_2 + k'_2) \dots (x_n + k'_n)$$

$$M_i = \prod_{j=1}^n (A_j F_{ij} + A'_j F'_{ij} + K'_j)$$



شکل (b ۱۸) حافظه شرکت پذیر m کلمه‌ای که هر کلمه دارای n سلول است.

■ Example

- A: Argument Register
- K: Key Register (If $K_i == 0$, comparison will be skipped)
- C: CAM cell
- M: Match Register

Argument Reg

A

1	0	1	1	0	0
---	---	---	---	---	---

Key Reg

K

0	1	1	0	0	0
---	---	---	---	---	---

Associative Memroy

1	1	0	1	1	1
0	0	1	1	1	1
1	1	1	1	1	1
1	0	0	1	1	0

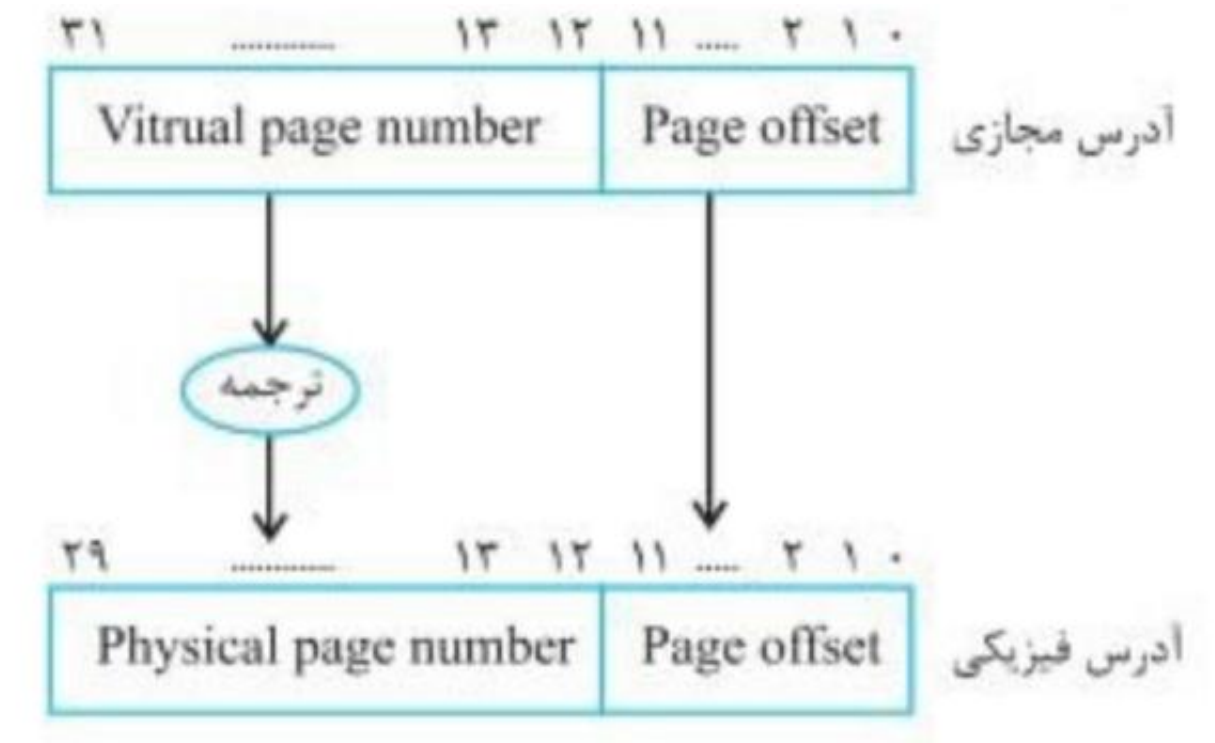
M : (Match Reg.)

0
1
0
0

- **Acts like cache**
- Acts as an extension of physical memory using disk storage
- Enables
 - Execution of programs larger than main memory
 - Better memory sharing between processes
- Benefits
 - Programs can be larger than physical **memory**
 - Improves efficiency by loading only necessary parts of the program
 - Enhances performance and memory management
- In modern systems, entire programs aren't loaded into memory at once
 - A program may have 100 pages, but only 10 might be active

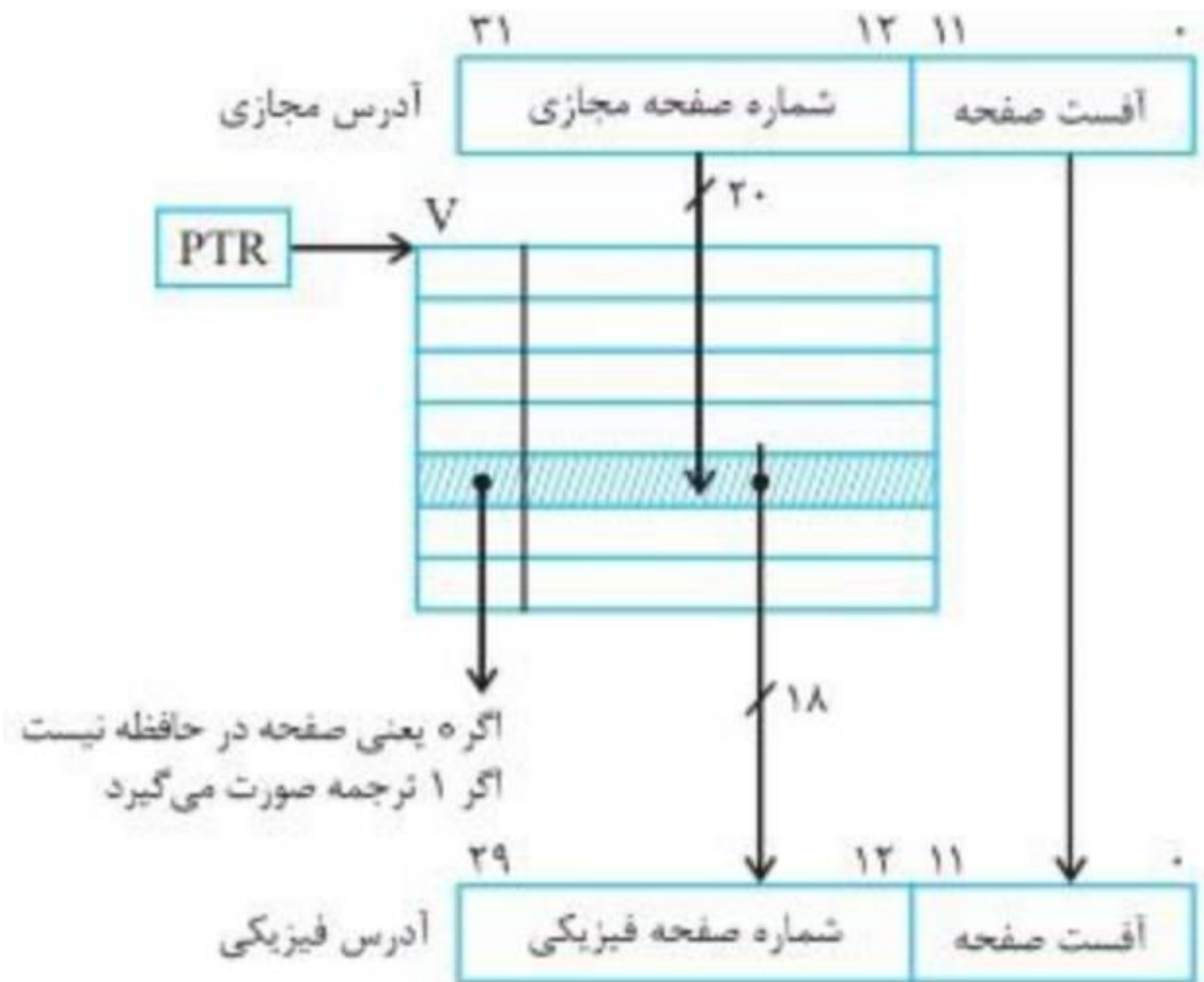
■ Virtual to Physical Address Translation

- Virtual memory addresses generated by the **CPU**
- Must be translated to physical addresses by **hardware and/or software**
- Address is divided into **two parts**
 - **Virtual Page Number (VPN)**
 - **Page Offset**
- Virtual Address = **[Virtual Page Number | Page Offset]**
 - **Page Offset remains constant** during translation
 - VPN is mapped to a **Physical Page Number (PPN)**
 - Final Physical Address = **[Physical Page Number | Page Offset]**



- **Virtual to Physical Address Translation (Cont'd)**
 - Translation of virtual addresses is done via a **Page Table**
 - The Page Table resides in main memory
 - Each entry in the table includes the physical page number (if present in memory)
 - **Valid bit (V)**: Indicates if the page is in memory
 - **PTR (Page Table Register)** points to the start of the page table in memory. It contains
 - Starting address of the page table
 - Total number of page table entries

■ Example

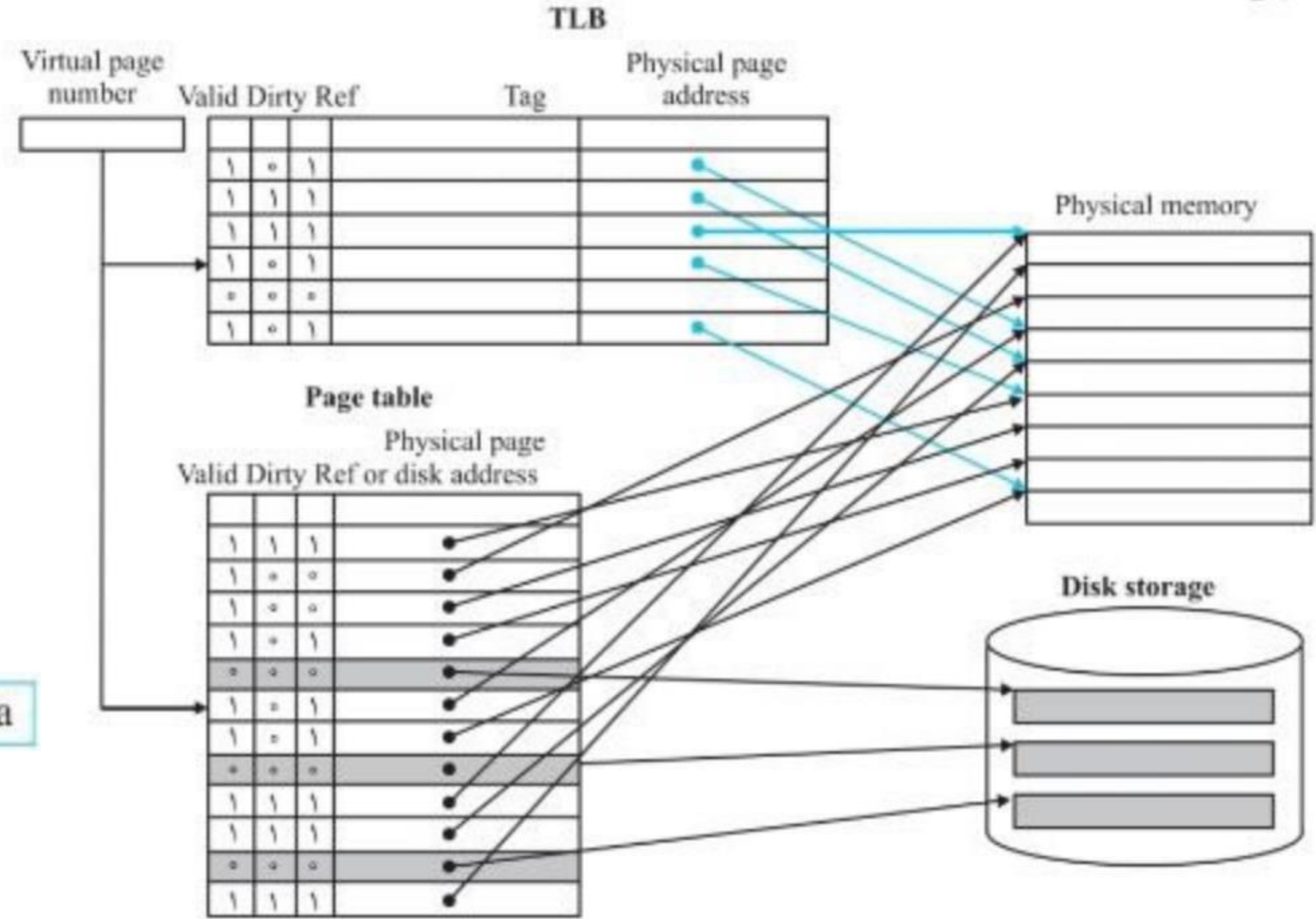


- **Virtual to Physical Address Translation (Cont'd)**
 - Each virtual memory access may require 2 memory accesses
 - One to fetch the page table entry
 - One to fetch the actual data from physical memory
 - Doubles access time
 - **TLB** fixes it (mostly)!
- **TLB**
 - TLB (Translation Lookaside Buffer) acts as a **cache** for recent address translations
 - Located **within the CPU**

■ How TLB works

- When a virtual address is issued, TLB is checked first for a match
 - If found => TLB hit => Direct access to physical memory
 - If not found => **Page Table** is accessed and then translation is added to the **TLB** for future accesses
- TLB Structure (Each row) includes
 - **Tag**: Part of the virtual address (identifies the page)
 - **Data**: Physical page address
 - **Control bits**
 - **Valid**: Entry is usable
 - **Dirty**: Page has been modified
 - **Ref** (Reference): Page has been accessed recently

■ TLB diagram:



■ TLB Row:

control	tag	data
---------	-----	------

■ Access Time (Latency)

■ For random access memory

- The time a read/write operation takes.
- From when address is introduced to the memory, until the data is accessible for use or is saved.

■ For non-random access memory

- The time it takes for read/write mechanism to move to the correct location.

■ Memory Cycle Time

- Only for random-access memories
- Access time + the time until the next access start.
- Is dependable on bus

■ Transfer rate

- The rate of data transfer from/to the memory
- For random access memory

$$\frac{1}{\text{cycle time}}$$

- For non-random access memory

$$T_N = T_A + \frac{N}{R}$$

T_n = average time to read/write N bits

T_A = average access time

R = transfer rate (bits per second)



Thanks
